



Ministerul Educației și Cercetării
Universitatea "OVIDIUS" Constanța
Facultatea de Matematică și Informatică
Specializarea Informatică

FlawFinder: Soluție AI pentru detectarea defectelor în producție

Lucrare de licență

Coordonator științific:
Conf. univ. dr. Pelican Elena

Absolvent:
Buzgaru Delia-Maria

Constanța
2026

Cuprins

Cuprins	i
Listă de figuri	iii
1 Abstract	2
1.1 Limba română	2
1.2 Limba engleză	3
2 Introducere și motivație	5
2.1 Introducere	5
2.2 Motivație	6
2.3 Starea actuală a domeniului	6
2.4 Obiectivul proiectului	7
3 Tehnologii utilizate, algoritmi și seturi de date	8
3.1 Tehnologii	8
3.1.1 Limbajul de programare	8
3.1.2 Bibliotecile PyTorch și Torchvision	8
3.1.3 Interfața grafică	9
3.1.4 Biblioteci de calcul	10

3.2	Algoritmi	10
3.2.1	Autoencoder	10
3.2.2	PaDiM	12
3.2.3	PatchCore	14
3.3	Seturi de date	17
4	Arhitectura aplicației	20
4.1	Arhitectura generală a sistemului	20
4.2	Procesarea datelor	25
4.3	Ingineria codului	25
4.4	Managementul modelelor	26
5	Prezentarea sistemului propus	27
5.1	Interfața grafică	27
5.2	Analiza individuală	34
5.3	Analiza unui grup	36
5.4	Stabilirea pragurilor și a algoritmului optim	38
6	Concluzii	44
6.1	Rezultate	44
6.2	Limitări ale sistemului	46
6.3	Perspective de viitor	47
6.4	Concluzie	47
	Bibliografie	49

Listă de figuri

3.1	Exemple de obiecte din MVTec AD	18
3.2	Circuit fără defecte	19
3.3	Circuit cu defecte	19
4.1	Diagrama cazurilor de utilizare	22
4.2	Diagrama de activități analiză individuală	23
4.3	Diagrama de activități analiză în grup	24
5.1	Fereastra în momentul deschiderii sistemului.	27
5.2	Lista cu componentele învățate de model.	28
5.3	Interfață pentru Autoencoder	29
5.4	Interfață pentru PaDiM	29
5.5	Interfață pentru PatchCore	29
5.6	Cele trei butoane ale sistemului propus.	31
5.7	Rezultatele din interfață pentru reconstrucția imaginii originale cu o componentă corectă.	32
5.8	Rezultatele din interfață pentru un algoritm cu hartă termică cu o componentă defectă.	33
5.9	Bara de stare în timpul rulării unei singure imagini	33
5.10	Bara de stare cu rezultatul unei rulări pentru o singură imagine.	34

5.11	Bara de stare în timpul rulării unui director	34
5.12	Bara de stare cu rezultatul rularii unul întreg director.	34
5.13	Interfața grafică a sistemului pentru generarea raportului de scoruri.	39
5.14	Exemplu de fișier text cu rezultatele scorurilor pentru PCB1 utilizând algoritmul Autoencoder.	40

Capitolul 1

Abstract

1.1 Limba română

În mediul industrial modern, producția a fost transformată de automatizarea și digitalizarea sa, dar până recent automatizarea a avut loc doar în cadrul asamblării. Controlul calității a fost un proces ce depindea de efortul uman, dar verificarea manuală necesită mult timp, iar oamenii pot lua decizii eronate din cauza oboselii sau a subiectivității. O soluție pentru această problemă a fost introducerea inteligenței artificiale în controlul calității. Pentru că învățarea profundă clasică necesită multe imagini cu produse defecte, lucru greu de obținut în cadrul unei fabrici ce produce majoritatea produselor perfecte, au fost introduși algoritmi ce învață cum trebuie să arate un obiect conform, orice variație de la acesta fiind clasificată ca defect.

În această lucrare a fost realizată implementarea unor modele nesupervizate de detectare a anomaliilor, obiectivul principal fiind realizarea sistemului propus sub forma unei aplicații software complete. Sistemul propus prezintă o interfață grafică intuitivă și ușor de înțeles, ce poate fi ulterior integrată în cadrul unui flux real de producție. În cadrul lucrării au fost implementați și comparați trei algoritmi utilizați actual în controlul calității: Autoencoder, PaDiM (Patch Distribution Modeling) și PatchCore. Algoritmul ce folosește un Autoencoder codifică și decodifică imaginile, acestea fiind reconstruite și analizate în funcție de calitatea imaginii noi. Pentru algoritmi PaDiM și PatchCore a fost utilizată rețeaua convoluțională preantrenată ResNet18 pentru extragerea trăsăturilor fiecărei componente. PaDiM se folosește de distribuția Gaussiană multivariată, și găsește anomalii cu ajutorul distanței Mahalanobis, în timp ce PatchCore adaugă o flexibilitate ce elimină alarmele false ce apar odată cu variațiile luminii.

În dezvoltarea sistemului propus au fost utilizate limbajul de programare Python și bibliotecile precum PyTorch și Torchvision pentru manipularea tensorilor și prelucrarea imaginilor de antrenare utilizate. Pentru partea matematică a sistemului au fost necesare bibliotecile NumPy, Scikit-learn și SciPy, iar matricea de antrenare a fost serializată cu pachetul Pickle pentru a optimiza timpii de execuție. Interfața grafică a fost realizată cu bibliotecile PyQt5 și CustomTkinter, ce oferă o bară de stare ce se actualizează în timp real și reprezentări vizuale

ale reconstrucțiilor create în Autoencoder și hărți termice din cadrul algoritmilor PaDiM și PatchCore.

Testarea propriu-zisă a fost realizată asupra a două seturi de date a câte patru tipuri de componente fiecare: jumătate din setul MVTec AD (cablu, șurub, piuliță, tranzistor) și cealaltă jumătate din setul de date SPot-the-Difference (patru tipuri de plăci de circuite imprimate). În urma antrenării și testării, rezultatele indică faptul că algoritmul PaDiM este optim pentru cinci din cele opt obiecte.

1.2 Limba engleză

In the modern industrial environment, production has been transformed by its automation and digitalization, but until recently, automation took place only within assembly. Quality control was a process that depended on human effort, but manual verification requires a lot of time, and humans can make erroneous decisions due to fatigue or subjectivity. A solution to this problem was the introduction of artificial intelligence in quality control. Because classical deep learning requires many images of defective products, which is difficult to obtain within a factory that produces mostly perfect products, algorithms were introduced that learn how a compliant object should look, any variation from it being classified as a defect.

In this thesis, unsupervised anomaly detection models were implemented, the main objective being the creation of the proposed system as a complete software application. The proposed system features an intuitive and easy-to-understand graphical interface, which can be subsequently integrated into a real production workflow. Within the thesis, three algorithms currently used in quality control were implemented and compared: Autoencoder, PaDiM (Patch Distribution Modeling), and PatchCore. The algorithm that uses an Autoencoder encodes and decodes the images, which are reconstructed and analyzed based on the quality of the new image. For the PaDiM and PatchCore algorithms, the pre-trained convolutional network ResNet18 was used to extract the features of each component. PaDiM uses the multivariate Gaussian distribution and finds anomalies with the help of the Mahalanobis distance, while PatchCore adds a flexibility that eliminates false alarms that appear along with lighting variations.

In the development of the proposed system, the Python programming language and libraries such as PyTorch and Torchvision were used for tensor manipulation and processing the training images used. For the mathematical part of the system, the NumPy, Scikit-learn, and SciPy libraries were necessary, and the training matrix was serialized with the Pickle package to optimize execution times. The graphical interface was created with the PyQt5 and CustomTkinter libraries, which offers a status bar that updates in real time and visual representations of the reconstructions created in the Autoencoder and heatmaps from the PaDiM and PatchCore algorithms.

The actual testing was performed on two datasets of four types of components each: half from the MVTec AD dataset (cable, screw, nut, transistor) and the other half from the

SPot-the-Difference dataset (four types of printed circuit boards). Following training and testing, the results indicate that the PaDiM algorithm is optimal for five out of the eight objects.

Capitolul 2

Introducere și motivație

2.1 Introducere

În procedeele industriale, automatizarea și digitalizarea au reprezentat schimbări radicale în modul în care un obiect este creat. În timp ce asamblarea propriu-zisă a fost deja automatizată cu mult timp în urmă, controlul calității a avut nevoie de o tehnologie mai avansată pentru a putea fi îmbunătățit.

Schimbarea aceasta impune integrarea unor sisteme tehnologice inteligente și capabile să ia decizii în timpul fluxului de asamblare asupra calității produselor pentru a asigura conformitatea componentelor, dar și pentru a optimiza timpul și resursele necesare procesului.

Înainte de avansul tehnologic actual, verificarea calității unui produs era o muncă ce necesita efort uman, unde se pierdeau multe ore verificând fiecare produs în parte. Oamenii pot fi subiectivi și obosesc după ore lungi de muncă, pierzându-și precizia. Din aceste motive, odată cu apariția inteligenței artificiale, industria a continuat să caute metode de a digitaliza acest proces.

În cadrul disciplinei „Algoritmi de calcul științific (Recunoașterea formelor)” [1], tehnicile clasice de recunoaștere facială studiate se bazează pe învățarea profundă supravegheată, ce are nevoie de imagini cu toate persoanele pe care trebuie să le recunoască. Astfel, într-un context de control al calității, sistemul propus necesita atât poze de antrenare conforme, cât și defecte pentru ca acesta să poată distinge între ele, dar acest lucru este greu de obținut într-o fabrică unde majoritatea produselor sunt fără defecte. Soluția aleasă a fost crearea unor modele ce sunt antrenate doar pe produse perfecte, clasificând ca defect orice anomalie ce depășește norma, această învățare nesupravegheată redefinind problema detecției.

2.2 Motivație

Motivația din spatele sistemului propus a fost dorința de a implementa acești algoritmi de inteligență artificială și de detectare a anomaliilor pe seturi de date diverse, într-un program complet, cu o interfață grafică simplă și ușor de folosit. La fiecare set de date au fost întâmpinate probleme diferite, alegerea unui singur algoritm universal perfect pentru orice fiind imposibilă.

Variațiile de scară și localizare a anomaliilor oferă o complexitate structurală ce motivează necesitatea unui sistem cu mai mult de un algoritm capabil să analizeze comportamentul fiecărui tip de componentă implementat. Componentele industriale au structuri complexe și foarte diverse, cu texturi și culori diferite ce pot deveni o piedică în procesul de analizare. Nevoia de a dezvolta un sistem ce poate fi adaptat asupra acestei diversități a reprezentat un scop principal al cercetării. Lucrarea a fost creată pentru a oferi aplicabilitate teoriei și tehnicilor avansate din domeniul controlului calității folosind inteligența artificială.

Scopul a fost analiza a diferiți algoritmi de detectare a anomaliilor și implementarea celor ce s-au dovedit optimi pentru seturile de date folosite, cât și să înțeleg mai mult despre domeniul controlului calității cu ajutorul inteligenței artificiale.

2.3 Starea actuală a domeniului

La momentul actual, domeniul de detectare a defectelor se bazează din ce în ce mai mult pe tehnologii de Deep learning ce nu necesită imagini cu defecte pentru a învăța. În prezent, cercetările s-au îndepărtat de algoritmii clasici de procesare a imaginilor, optând pentru învățarea nesupravegheată. Aceasta permite sistemului propus să înțeleagă geometria și detaliile unei componente, marcând defectele ca anomalii.

Tehnologiile de top din prezent se clasifică în diferite categorii, cum ar fi:

1. Tehnologiile ce încearcă reconstrucția imaginii de test inițiale, aceasta fiind clasificată ca defectă dacă sistemul propus a generat o eroare în procesul de reconstrucție. Un exemplu al acestui caz fiind Autoencoderul folosit în cadrul aplicației, ce codifică și mai apoi decodifică imaginile de test. Această abordare comprimă imaginea de intrare într-un spațiu latent cu dimensiuni reduse prin straturi convoluționale succesive.
2. Tehnologii ce se folosesc de rețele neuronale preantrenate pentru a detecta trăsături subtile ale obiectului. Cei doi algoritmi PaDiM și PatchCore, integrați în sistemul propus, se încadrează în această categorie, aceștia folosind rețeaua neuronală ResNet18 și mai apoi calculând distanța dintre trăsăturile imaginii de test față de cele învățate. PaDiM se folosește de distribuții Gaussiene multivariate și măsoară anomalia cu distanța Mahalanobis, în timp ce algoritmul PatchCore optimizează învățarea trăsăturilor cu ajutorul unei bănci de memorie.

2.4 Obiectivul proiectului

Scopul principal al sistemului propus a fost acela de a realiza un software complet, cu o interfață intuitivă, ce ar putea fi integrat în interiorul unei fabrici. S-a realizat integrarea a trei algoritmi diferiți (Autoencoder, PaDiM și PatchCore) și am trecut atât prin antrenarea obiectelor pe fiecare algoritm, cât și prin testarea și evaluarea algoritmului optim în funcție de setul de date.

Proiectul a fost dezvoltat după principiul modularității, unde procesele de backend sunt izolate de interfața grafică. De asemenea, fiecare algoritm are un script propriu, separat de cel de antrenare. Această arhitectură menține stabilitatea sistemului propus și facilitează posibilitatea de a introduce noi algoritmi în viitor.

Un alt obiectiv tehnic a fost alegerea pragurilor de decizie. Deoarece sistemele nesupravegheate nu au o metodă de calcul a acestui prag, a fost necesară dezvoltarea unui software secundar cu rol de a calcula și crea rapoarte text cu minimul, maximul, media și o listă cu toate scorurile imaginilor din directoarele alese.

Scopul a fost ca aplicația rezultată să poată fi ulterior modificată și conectată la un caz real, într-o fabrică, și să nu rămână doar la stadiul teoretic, motiv ce a dus la adăugarea opțiunii analizei în grup, analiza individuală nefiind atât de utilă cu un flux foarte rapid de producție. Prin această automatizare a sortării, sistemul devine aplicabil în mediul industrial.

De asemenea, s-a urmărit ca interfața grafică să fie simplă pentru persoanele care vor lucra în fabrică și să nu fie obositoare pentru ochi, motiv pentru care a fost folosit un fundal închis la culoare. Structura interfeței elimină barierele tehnice de utilizare, aceasta fiind foarte intuitivă pentru orice nou utilizator.

Contribuțiile practice și teoretice ale autorului în realizarea acestui sistem de control al calității sunt:

- ▷ **Implementarea și compararea a trei algoritmi de detecție a anomaliilor** – Au fost implementate și adaptate arhitecturi diverse (Autoencoder, PaDiM și PatchCore) în cadrul sistemului propus.
- ▷ **Dezvoltarea unei aplicații software complete pentru controlul calității** – Aplicația realizată este completă, stabilă și aplicabilă în fluxul industrial.
- ▷ **Analiza experimentală pe seturi de date industriale** – Performanțele algoritmilor au fost evaluate asupra a opt clase de obiecte cu grade diferite de complexitate.
- ▷ **Determinarea empirică a pragurilor de decizie** – Pentru determinarea pragurilor de eroare, a fost conceput un software secundar pentru analiza statistică a scorurilor.

Capitolul 3

Tehnologii utilizate, algoritmi și seturi de date

3.1 Tehnologii

3.1.1 Limbajul de programare

Limbajul de programare utilizat în scrierea sistemului propus a fost Python, deoarece acesta este standardul în domeniul Inteligenței Artificiale și oferă o multitudine de biblioteci create special pentru machine learning. Acesta are avantajul unei interfețe simple, apropiată limbajului natural, ce dă libertate programatorului să se concentreze asupra calculelor matematice complexe utilizate în machine learning.

De asemenea, Python este un limbaj flexibil și dinamic, ce reprezintă un mediu optim pentru cercetarea și experimentarea specifică proiectelor de Inteligență Artificială. Memoria este manageriată automat cu ajutorul mecanismelor integrate de colectare a reziduurilor, fapt ce elimină nevoia de a aloca manual resurse hardware și reduce apariția erorilor. Python permite testarea rapidă a fișierelor individuale de cod, ușurând acționarea rapidă asupra erorilor ce sunt inevitabil întâlnite în faza de proiectare a sistemului propus.

3.1.2 Bibliotecile PyTorch și Torchvision

Biblioteca PyTorch este framework-ul principal al sistemului propus. Acesta a fost necesar pentru manipularea tensorilor (`torch.Tensor`), definirea arhitecturii (`torch.nn`), funcții matematice (`torch.nn.functional`), optimizare (`torch.optim`) și mutarea calculelor de pe procesor (CPU) pe placa grafică (CUDA)[2]. Torchvision, extensia lui PyTorch, a fost folosită pentru a ușura manipularea imaginilor. În proiect, ea a fost utilizată pentru a descărca modelele preantrenate pentru PaDiM și PatchCore (`torchvision.models`) și pentru standardizarea imaginilor (`torchvision.transforms` și `transforms.ToTensor()`).

Din pachetul `torch.nn` au fost utilizate funcții (`BatchNorm2d` și `GroupNorm`) specializate de normalizare, cât și funcții de activare (`LeakyReLU` și `GELU`[3]). De asemenea, am utilizat optimizatorul Adam împreună cu funcția `CosineAnnealingLR`, cu scopul de a ajusta rata de învățare în timpul etapei de antrenare a sistemului propus.

3.1.3 Interfața grafică

Pentru interfața grafică a fost folosită biblioteca `CustomTkinter`[4], datorită suportului său pentru teme vizuale întunecate și design minimalist ce previne oboseala vizuală a operatorilor. Elementele din fereastra principală sunt structurate modular, cu butoane utilizate pentru inițializarea proceselor (`ctk.CTkButton`) și etichete menite să explice fiecare parte a sistemului (`ctk.CTkLabel`). De asemenea, a fost utilizată o listă de selecție a componentelor cu ajutorul funcției `ctk.CTkComboBox`.

Interfața grafică este bazată pe evenimente; astfel, atunci când operatorul selectează o imagine, interfața preia calea cu „filedialog” și rulează asincron analiza, actualizând în același timp zona rezultatelor cu harta termică sau reconstrucția. Bara de stare oferă informații imediate în timpul rulării analizei pentru a asigura fluiditatea sistemului și a asigura utilizatorul că fereastra nu este blocată.

3.1.4 Biblioteci de calcul

Pentru partea matematică a algoritmilor au fost utilizate biblioteci fundamentale precum:

1. **NumPy**[5]: Este un pachet de bază pentru calcule științifice în Python. Această bibliotecă oferă suport pentru tablouri multidimensionale de mare densitate (ndarrays) și funcții matematice optimizate. În cadrul sistemului propus, NumPy a fost folosit pentru funcții precum `np.mean()` pentru calcularea mediei, `np.std()` pentru a determina deviația standard și `np.cov()` pentru calculul matricei de covarianță.
2. **Scikit-learn**[6]: Aceasta este o bibliotecă foarte utilizată în Machine Learning pentru implementări eficiente de algoritmi și metrici de evaluare. Pachetul `sklearn.metrics` a fost folosit pentru determinarea ariei de sub curba de caracteristici de operare.
3. **SciPy**[7]: A fost construită pe baza bibliotecii NumPy pentru a extinde posibilitățile calculului științific cu module dedicate algebrei liniare, optimizării și statisticii matematice. Din biblioteca SciPy am folosit funcția `mahalanobis`, utilizată în algoritmul PaDiM.
4. **Pickle**: Este un modul nativ din Python, conceput pentru serializarea și deserializarea structurilor de obiecte complexe. În sistemul propus, pachetul a fost utilizat pentru serializarea și salvarea matricelor de medii și de covarianță din antrenare, ce previne nevoia de a le calcula la fiecare rulare.

Aceste biblioteci matematice au fost utilizate împreună pentru a uni capacitatea de calcul brută cu analize statistice avansate. Folosirea lor a permis transferul rapid de date între algoritmii de analiză și straturile de învățare profundă (deep learning) și a redus consumul memoriei și timpul de execuție.

3.2 Algoritmi

3.2.1 Autoencoder

Primul algoritm implementat în sistemul propus este autoencoderul. În cadrul proiectului, a fost utilizat un autoencoder subcomplet, adaptat după arhitectura generală descrisă în cartea „Deep Learning” de Ian Goodfellow, Yoshua Bengio și Aaron Courville [8]. Reducerea spațiului latent la 1024 forțează modelul să învețe doar trăsăturile importante, ignorând zgomotul. De asemenea, cele patru straturi convoluționale succesive, ce cresc numărul de canale (32, 64, 128, 256), reprezintă un avantaj computațional în comparație cu un autoencoder cu un singur strat. Această structură permite sistemului să învețe atât trăsături de frecvență înaltă, cu ajutorul primelor straturi, cât și elemente mai subtile, învățate în straturile adânci.

Listing 3.1: Definirea encoderului.

```

# Extras din implementarea clasei Autoencoder
def __init__(self, width, height, depth, filtre=(32, 64, 128,
256), DimLatenta=1024):
    super(Autoencoder, self).__init__()

    layers_e = []
    canal_curent = depth
    for f in filtre:
        layers_e.append(nn.Conv2d(canal_curent, f, kernel_size=3, stride
=2, padding=1))
        layers_e.append(nn.LeakyReLU(0.2))
        layers_e.append(nn.BatchNorm2d(f))
        canal_curent = f

    self.formaOriginala = (filtre[-1], height // (2**len(filtre)),
        width // (2**len(filtre)))
    total_trasaturi = np.prod(self.formaOriginala)

    self.encoder = nn.Sequential(
        nn.Sequential(*layers_e),
        nn.Flatten(),
        nn.Linear(total_trasaturi, DimLatenta)
    )
    # ... (restul decodorului) ...

    def forward(self, x):
        cod_latent = self.encoder(x)
        reconstructie = self.decoder(cod_latent)
        return reconstructie

```

În funcția de codificare, imaginea primită este înjumătățită repetat prin intermediul straturilor `nn.Conv2d` cu un pas (stride) de doi. Astfel, sistemul procesează hărțile de caracteristici sărind din doi în doi pixeli, aducând rezultatul la o mărime de $2^4 = 16$ ori mai mică decât cea inițială. Funcțiile `nn.LeakyReLU` și `nn.BatchNorm2d` au rolul de a preveni stagnarea și a stabiliza antrenarea. Rezultatul este ulterior aplatizat într-un vector unidimensional cu `nn.Flatten()`, urmând ca acesta să fie trimis către decodor.

În faza de decodificare, operațiile sunt realizate invers față de codificare. Vectorul latent este inițial remodelat la forma tridimensională (`self.formaOriginala`). Imaginea este adusă înapoi la forma originală folosind straturi de convoluție transpusă (`nn.ConvTranspose2d`), ce inserează spații între pixeli și aplică filtre învățabile.

Listing 3.2: Definirea și logica internă a decodorului.

```

self.decoder_linear = nn.Linear(DimLatenta, total_trasaturi)

layers_d = []
filtre_inverse = list(filtre[::-1])
for i in range(len(filtre_inverse) - 1):
    layers_d.append(nn.ConvTranspose2d(filtre_inverse[i],
        filtre_inverse[i+1], kernel_size=3, stride=2, padding=1,
        output_padding=1))

```

```

layers_d.append(nn.LeakyReLU(0.2))
layers_d.append(nn.BatchNorm2d(filtre_inverse[i+1]))

layers_d.append(nn.ConvTranspose2d(filtre_inverse[-1],
    filtre_inverse[-1], kernel_size=3, stride=2, padding=1,
    output_padding=1))
layers_d.append(nn.LeakyReLU(0.2))

self.decoder_conv = nn.Sequential(*layers_d)

self.strat_final = nn.Sequential(
nn.Conv2d(filtre_inverse[-1], depth, kernel_size=3, padding=1),
nn.Sigmoid()
)

def decoder(self, z):
x = self.decoder_linear(z)
x = x.view(-1, *self.formaOriginala)
x = x.view(-1, *self.formaOriginala)
x = self.decoder_conv(x)
x = self.strat_final(x)
return x

```

La final este folosită o convoluție standard pentru a mapa numărul de canale înapoi la valoarea inițială. Este aplicată funcția `nn.Sigmoid()`, ce aduce valorile pixelilor în intervalul $[0, 1]$ pentru ca aceasta să fie similară cu imaginea de intrare normalizată.

Pentru antrenarea rețelei a fost utilizată eroarea medie pătratică (MSE Loss - Mean Squared Error) calculată cu pixelii imaginii reconstruite și pixelii imaginii originale. În faza de testare, funcția este o metrică brută de anomalie, unde zonele ce prezintă abateri geometrice sau texturi necunoscute sistemului generează o valoare ridicată a erorii de reconstrucție.

3.2.2 PaDiM

Al doilea algoritm s-a dovedit a fi și cel mai eficient în detectarea anomaliilor. PaDiM (Patch Distribution Modeling) a fost algoritmul optim ales pentru cinci din cele opt seturi de date. În lucrarea științifică „PaDiM: a Patch Distribution Modeling Framework for Anomaly Detection and Localization” [9], Thomas Defard, Aleksandr Setkov, Angélique Loesch și Romaric Audigier introduc pentru prima dată conceptul de modelare a distribuției patch-urilor. Ei detaliază că metodele existente la momentul lucrării întâlneau probleme de scalabilitate, precum un timp de antrenare foarte mare sau faptul că algoritmul KNN (K-Nearest Neighbour) trebuie să compare la fiecare rulare poza de test cu toate pozele de antrenare, ceea ce este prea lent pentru seturi de date industriale.

PaDiM rezolvă aceste obstacole folosind o rețea convoluțională neuronală preantrenată (CNN). Algoritmul se folosește de o distribuție Gaussiană multivariată ce funcționează cu doi parametri: media (μ), ce stabilește centrul unei piese fără defecte, și matricea de covarianță (Σ), care salvează variația și modul în care aceste variații interacționează una cu alta. O altă trăsătură a algoritmului PaDiM este corelarea dintre diferite niveluri semantice; sistemul nu

folosește doar ultimul strat din rețea, folosește trei straturi diferite ce pot identifica atât detalii mari (forma obiectului, părți lipsă), cât și detalii fine (zgârieturi, defecte subtile).

Listing 3.3: Extragerea trăsăturilor și antrenarea în PaDiM.

```
def extrage_feature_map(self, x):
    outputs = []
    def hook(module, input, output):
        outputs.append(output)

    h1 = self.model.layer1.register_forward_hook(hook)
    h2 = self.model.layer2.register_forward_hook(hook)
    h3 = self.model.layer3.register_forward_hook(hook)

    with torch.no_grad():
        self.model(x)

    h1.remove(); h2.remove(); h3.remove()

    for i in range(3):
        outputs[i] = torch.nn.functional.interpolate(outputs[i], size
            =(32, 32), mode='bilinear', align_corners=False)

    return torch.cat(outputs, 1) # Rezultat tensor: [Banc, 448, 32,
        32]

def train(self, dataloader):
    features_list = []
    for batch in dataloader:
        batch = batch.to(self.device)
        f_map = self.extrage_feature_map(batch)
        features_list.append(f_map.cpu())

    features = torch.cat(features_list, 0).numpy() # [N, C, H, W]
    n, c, h, w = features.shape
    features = features.reshape(n, c, h * w)

    self.mean = np.mean(features, axis=0)
    self.inv_covariance = np.zeros((h * w, c, c))
    identity = np.identity(c)

    for i in range(h * w):
        self.inv_covariance[i] = np.linalg.inv(np.cov(features[:, :, i],
            rowvar=False) + 0.01 * identity)
```

Rețeaua neuronală preantrenată ResNet18, folosită în cadrul sistemului propus pentru procesul de extragere a hărților de caracteristici, are la bază arhitectura introdusă în lucrarea academică „Deep Residual Learning for Image Recognition” [10]. În sistemul de față, modelul PaDiM extrage informații din trei blocuri ale rețelei, ale căror caracteristici au rezoluții diferite. Din acest motiv, este necesară fuziunea acestora prin redimensionarea lor cu ajutorul interpolării biliniare. În final, hărțile sunt concatenate, rezultând un vector de caracteristici pentru fiecare regiune (patch) a imaginii.

Harta de caracteristici este generată de cele trei straturi ale rețelei ResNet18 prin înregistrarea unor mecanisme de interceptare a semnalelor. Astfel este capturată starea internă a rețelei în timpul trecerii imaginii prin straturi, evitând perturbarea calculului gradientului. Hărțile sunt aduse la o rezoluție uniformă de 32×32 pixeli și concatenate pe axa canalelor prin interpolare biliniară, generând un tensor final stabil de 448 de dimensiuni pentru fiecare punct spațial.

Listing 3.4: Calculul distanțelor în funcția predict

```
def predict(self, x):
    f_map = self.extract_feature_map(x).cpu().numpy()[0]
    c, h, w = f_map.shape
    f_map = f_map.reshape(c, h * w).T # [H*W, C]

    distances = []
    for i in range(h * w):
        dist = mahalanobis(f_map[i], self.mean[:, i], self.inv_covariance[i])
        distances.append(dist)

    anomaly_map = np.array(distances).reshape(h, w)
    score = np.max(anomaly_map)
    return score, anomaly_map
```

În secvența de cod de mai sus, este utilizată distanța Mahalanobis[11], ce măsoară cât de diferit e fiecare pixel din imaginea de testare față de ceea ce modelul a învățat că este normal în antrenare.

$$D(x) = \sqrt{(x - \mu)^T \Sigma^{-1} (x - \mu)} \quad (3.1)$$

Spre deosebire de distanța Euclidiană, distanța Mahalanobis utilizată în sistemul propus ține cont de variabilitatea canalelor semantice și de corelația dintre trăsăturile extrase de rețeaua neuronală ResNet18 folosită. La final, scorul este ales în funcție de cea mai mare distanță salvată.

3.2.3 PatchCore

Ultimul algoritm utilizat în sistemul propus, PatchCore, este un succesor al lui PaDiM. În lucrarea științifică „Towards Total Recall in Industrial Anomaly Detection” [12], autorii introduc PatchCore și îl diferențiază de PaDiM prin flexibilitatea sa. În timp ce PaDiM, prin utilizarea distanței Mahalanobis, este rigid și necesită alinierea obiectelor într-o poziție exactă, PatchCore creează o bancă de memorie globală pe care o pot accesa toate secțiunile verificate în etapa de testare. De asemenea, pentru algoritmul PatchCore, nu este necesar ca imaginile să aibă aceeași formă și dimensiune.

Pentru a implementa algoritmul în aplicație, a fost integrată rețeaua neuronală preantrenată ResNet18, similar algoritmului PaDiM menționat în secțiunea anterioară. În model este introdusă o adaptare neuronală (`self.adaptor`), diferită de o bancă de memorie statică

simplă. Acest modul de adaptare constă într-o secvență de straturi convoluționale, normalizări de grup (`nn.GroupNorm`) și funcții de activare de tip GELU (Gaussian Error Linear Unit). Normalizarea de grup este folosită în locul celei de lot (`BatchNorm`), deoarece aduce o independență față de mărimea lotului de date (batch size).

Listing 3.5: Procesul de antrenare

```
def train_model(self, dataloader, epochs=50, lr=2e-4):
    optimizer = torch.optim.AdamW(
        self.model.adaptor.parameters(),
        lr=lr, weight_decay=1e-4
    )
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
        optimizer, T_max=epochs
    )

    self.model.train()
    self.model.feature_extractor.eval()

    for epoch in range(epochs):
        total_loss = 0
        for batch in dataloader:
            batch = batch.to(self.device)
            optimizer.zero_grad()
            z, _ = self.model(batch)
            loss = self.model.compute_loss(z)
            loss.backward()

            torch.nn.utils.clip_grad_norm_(
                self.model.adaptor.parameters(), max_norm=1.0
            )
            optimizer.step()
            total_loss += loss.item()

        scheduler.step()

    print("\nCalibrare statistici pe datele de training...")
    self._calibreaza_statistici(dataloader)

    def _calibreaza_statistici(self, dataloader):
        self.model.eval()
        scoruri = []
        with torch.no_grad():
            for batch in dataloader:
                batch = batch.to(self.device)
                z, _ = self.model(batch)
                anomaly_map = torch.mean(z**2, dim=1)
                scor_per_imagine = anomaly_map.mean(dim=(1, 2))
                scoruri.extend(scor_per_imagine.cpu().numpy().tolist())

        self.train_mean = float(np.mean(scoruri))
        self.train_std = float(np.std(scoruri)) + 1e-8
```

În momentul optimizării, modelul PatchCore folosește algoritmul AdamW[13] și scade rata de învățare folosind cosinus (CosineAnnealingLR)[14]. În final, algoritmul cali-

breață statisticile peste setul de date pentru a putea stoca valorile mediei și deviației standard utilizate în calibrarea finală.

În implementarea algoritmului PatchCore a fost introdus modulul de adaptare antrenabil (`self.adaptor`), format dintr-o succesiune de straturi convoluționale bidimensionale și normalizări pe grupuri `GroupNorm`[15]. Modelul rafinează caracteristicile peste extractorul `ResNet18`. Comparativ cu o bancă de memorie statică ce necesită spațiu mare de stocare și timp lung de căutare, acest adaptor învață să proiecteze hărțile de trăsături pe un spațiu optimizat, unde deviațiile pătratice pot fi direct evaluate.

În etapa de testare, sistemul calculează harta defectelor și un scor final al anomaliei în interiorul funcției `predict`.

Listing 3.6: Funcția `predict` de calculare a scorului în PatchCore

```
def predict(self, input_tensor):
    self.model.eval()
    with torch.no_grad():
        z, _ = self.model(input_tensor)

        anomaly_map = torch.mean(z**2, dim=1).unsqueeze(1)
        anomaly_map = F.interpolate(
            anomaly_map, size=(256, 256),
            mode='bilinear', align_corners=False
        )
        anomaly_map_np = anomaly_map.squeeze().cpu().numpy()

        scor_brut = float(np.mean(anomaly_map_np))

        if self.train_mean is not None:
            scor_final = (scor_brut - self.train_mean) / self.train_std
        else:
            scor_final = scor_brut

    return scor_final, [anomaly_map_np]
```

Operația `torch.mean(z**2, dim=1)` calculează deviația fiecărui pixel față de modelul normal învățat în antrenare, valorile mari reprezentând un defect sever. Harta rezultată este apoi redimensionată la formatul standard de 256×256 pixeli (`F.interpolate()`), urmând apoi să fie calculată media valorilor din harta de anomalii (scorul brut). Scorul final este calculat cu formula de mai jos, prin scăderea din scorul brut a mediei pieselor bune și împărțirea rezultatului la deviația standard a pieselor.

$$S_{\text{final}} = \frac{S_{\text{brut}} - \mu_{\text{train}}}{\sigma_{\text{train}}} \quad (3.2)$$

Scorul brut este mapat pe o distribuție standardizată ce a fost calibrată anterior pe un set de date cu obiecte fără defecte. Astfel, algoritmul devine eficient împotriva alarmelor false create de diferențe în modul în care lumina lovește componentele.

3.3 Seturi de date

În sistemul propus au fost utilizate două seturi de date a câte 4 obiecte diferite pentru antrenare. Aceste seturi de date apar în următoarele studii: „MVTec AD — A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection” [16] și „SPot-the-Difference Self-supervised Pre-training for Anomaly Detection and Segmentation” [17].

Aceste două seturi de date au fost alese datorită complexității și diversității necesare pentru evaluarea algoritmilor aleși. Cele patru componente din cadrul setului MVTec AD introduc obiecte cu structuri compacte, simple, dar cu dificultăți generate de reflexiile materialelor metalice, precum șurubul și piulița. În comparație, cele patru tipuri de plăci de circuite imprimate din setul de date SPot-the-Difference au o mulțime de detalii și componente mici ce pot fi greu de înțeles pentru sistemul propus. Utilizarea acestor seturi de date oferă o imagine mai bună asupra funcționalității algoritmilor de detectare ai anomaliilor selectați în cadrul sistemului propus.

De la MVTec AD au fost selectate următoarele obiecte: cablu, șurub, piuliță și tranzistor. Setul lor de date oferă imagini cu obiecte perfecte pentru antrenare, cât și imagini cu obiecte defecte pentru testare. Aceștia au sortat imaginile în funcție de defecte: zgârieturi, părți îndoite, părți lipsă, defecte de colorație etc. În lucrarea publicată, autorii au scris că, până la momentul acela, seturile de date pentru detectarea anomaliilor erau puține și se concentrau pe texturi, nu pe obiecte propriu-zise [16].

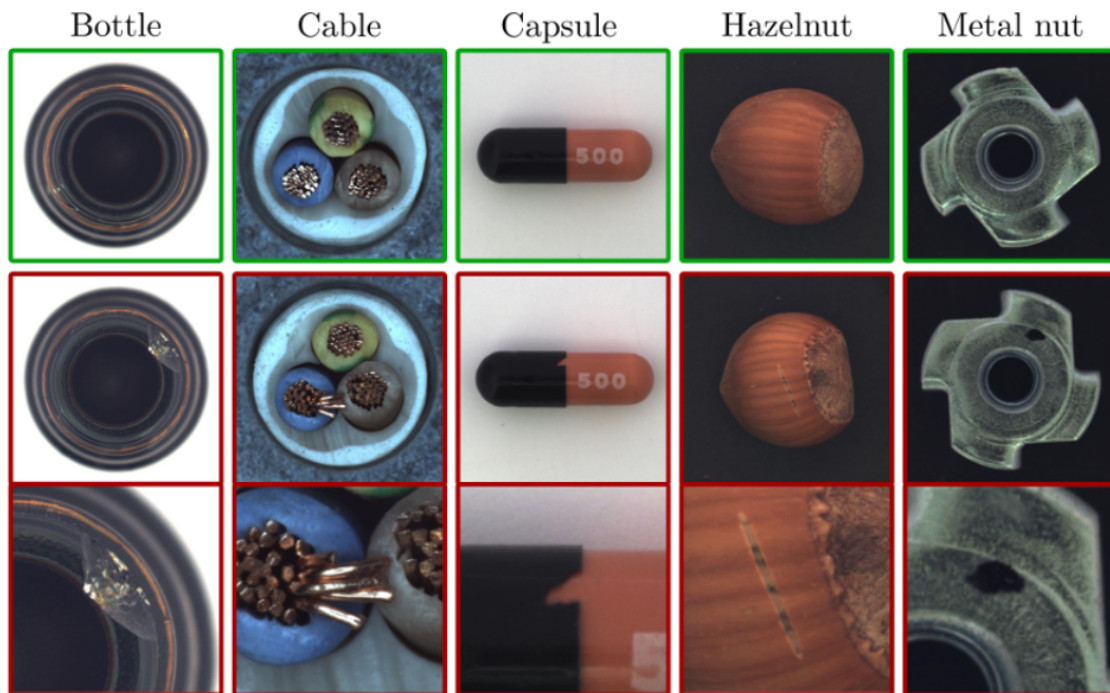


Figura 3.1: Exemple de obiecte din MVTec AD

Figura 3.1: Imaginea de mai sus conține exemple de obiecte din setul de date MVTec AD, unde pe primul rând sunt obiecte intacte, pe al doilea rând sunt obiecte defecte, iar pe al treilea rând imaginea este mărită pe zona exactă a defectului.

Imaginile din acest set de date sunt capturate în condiții de iluminare controlată, similară de la o componentă la alta, și cu fundaluri monocrome pentru a menține atenția asupra obiectului propriu-zis. O trăsătură importantă a acestui set de date este distribuția asimetrică a defectelor. Seturile conțin anomalii generate atât în timpul fabricației, cât și prin manipulare fizică. Defectele mici, precum zgârieturi sau găuri în izolația cablurilor, nu modifică foarte mult distribuția pixelilor în comparație cu defecte majore, precum o piesă lipsă. Acest set de date testează dacă algoritmii selectați sunt capabili să detecteze atât schimbări majore, cât și detalii subtile ale componentelor.

O provocare statistică în setul de date MVTec AD este reflexia de pe suprafețele metalice sub surse de lumină. Această problemă a fost în special întâlnită în cazul componentelor șurub și piuliță, piese complet metalice. Pentru aceste obiecte, Autoencoderului i-a fost foarte greu să identifice defectele, în schimb algoritmul PaDiM s-a dovedit optim datorită modelării sale Gaussiene multivariate și a distanței Mahalanobis. Astfel, sistemul propus învață să ignore reflexiile, ele fiind catalogate drept variații normale ale procesului.

Din al doilea set de date [17] au fost utilizate patru tipuri diferite de PCB-uri (plăci de circuite imprimate). Setul de date a avut mult mai multe imagini de antrenare decât cel de la MVTec, oferind o antrenare mai lentă, dar mai exactă în unele cazuri. În lucrarea publicată, autorii menționează că scopul lor a fost să introducă seturi de date cu caracteristici noi ce

nu au apărut în seturi de date anterioare. Cele patru tipuri de circuite reprezintă obiecte complexe, cu multe elemente de culori și forme diferite.

Setul de date SPot-the-Difference are o complexitate crescută, generată de multitudinea de componente, îmbinări și marcaje de pe suprafața plăcilor. De asemenea, zonele metalice sudate introduc reflexii ce pot genera alarme false, similare cu șuruburile și piulițele din setul de date anterior. O parte din defectele prezente în acest set de date sunt componente inversate, condensatori lipsă sau trasee întrerupte. Învățarea acestor plăci de circuite solicită o bază de antrenare mai mare și o precizie ridicată a reconstrucției Autoencoderului.

Această structură complexă pe care o prezintă plăcile de circuite imprimate pune în dificultate algoritmi rigizi din punct de vedere spațial. Deoarece PaDiM mapează distribuțiile Gaussiene pe coordonate fixe ale pixelilor, orice diferență de aliniere poate duce la erori în clasificare. Algoritmul PatchCore a fost superior asupra acestui set de date datorită flexibilității sale.

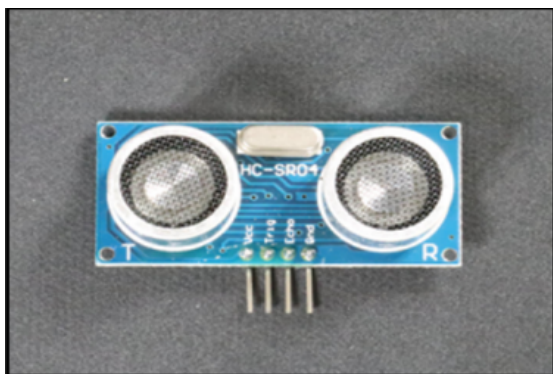


Figura 3.2: Circuit fără defecte



Figura 3.3: Circuit cu defecte

Comparând cele două seturi de date, poate fi observat că setul SPot-the-Difference oferă un volum mai mare de imagini de antrenare, ceea ce este ideal pentru modele precum Auto-encoderul, ce învață exclusiv pe setul de date oferit. Totuși, în contexte în care timpul și volumul imaginilor de antrenare sunt reduse, cum a fost în cazul setului de date MVTec AD, algoritmi bazați pe extractor și rețele neuronale preantrenate prezintă un avantaj.

Capitolul 4

Arhitectura aplicației

4.1 Arhitectura generală a sistemului

În acest capitol este reprezentată arhitectura sistemului propus și modul în care utilizatorii pot interacționa cu ea. Pentru a defini clar funcționalitatea și modul de operare al sistemului, au fost utilizate diagrame modelate în Astah, studiate în cadrul cursului „Ingineria sistemelor software” [18].

Prima diagramă importantă este diagrama cazurilor de utilizare. În această diagramă se pot observa actorii principali ai sistemului și acțiunile pe care le pot întreprinde asupra sistemului:

1. **Operatorul de calitate** se ocupă cu analiza componentelor. Acesta are acces la selecția tipului de componentă testată, încărcarea imaginilor de test și poate alege fie să verifice calitatea unei singure imagini, fie a unui grup întreg dintr-un director. Operatorul poate acționa doar asupra interfeței grafice, accesul la procesul de antrenare și la fișierele din spate fiind interzis.
2. **Administratorul** se ocupă cu partea din spate (backend-ul) a sistemului. În momentul în care este necesară introducerea unui nou set de date pentru un nou obiect în cadrul sistemului, administratorul poate folosi funcția de antrenare asupra acelui set de date. De asemenea, el are acces la interfața ce generează rapoartele scorurilor pentru a seta un prag de eroare al algoritmilor.

Pentru a înțelege mai bine felul în care utilizatorul interacționează cu procesele și modul în care acestea rulează în cadrul sistemului, au fost realizate două diagrame de activități. Acestea au rolul de a analiza în detaliu fiecare etapă a unui proces. În diagramele de mai jos poate fi observat fluxul operațiunilor, fiind separate clar pe coloane acțiunile utilizatorului față de acțiunile sistemului, ca urmare a inputului dat de utilizator.

În prima diagramă de activitate este definit procesul analizei unei singure imagini de testare. Actorul principal este operatorul de calitate, ce are rolul de a selecta tipul de componentă pe care urmează să o testeze și de a încărca imaginea de test înainte de a apăsa butonul de executare a analizei. Odată ce butonul „Execută analiza” este apăsat, sistemul verifică dacă a fost sau nu încărcată o imagine de test, iar în caz contrar afișează un mesaj de eroare. Dacă imaginea a fost încărcată, sistemul o trece prin etapa de preprocesare, rulează algoritmul ales ca fiind optim asupra ei, compară scorul obținut cu pragul de eroare setat și, în final, afișează verdictul împreună cu imaginea reconstruită sau harta termică.

Pentru cea de-a doua diagramă de activitate a fost analizat procesul de verificare a unui întreg director de imagini. Această acțiune începe similar cu analiza individuală: operatorul selectează tipul de componentă ce urmează să fie verificat, după care apasă butonul de „Testare în grup”, ce deschide o ferestrească de unde utilizatorul poate alege directorul cu imagini. Inițial, sistemul creează cele două directoare, „corect” și „defect”, în care vor fi sortate imaginile. Sistemul verifică după fiecare iterație dacă mai există imagini neprocesate, iar în caz afirmativ, încarcă imaginea curentă și rulează algoritmul optim asupra ei. În final, scorul obținut este comparat cu pragul de anomalie, iar imaginea este copiată în directorul „corect” sau „defect”, în funcție de rezultat. În momentul în care sistemul nu mai găsește imagini neprocesate, procesul este finalizat, iar rezultatele sunt afișate în bara de stare.

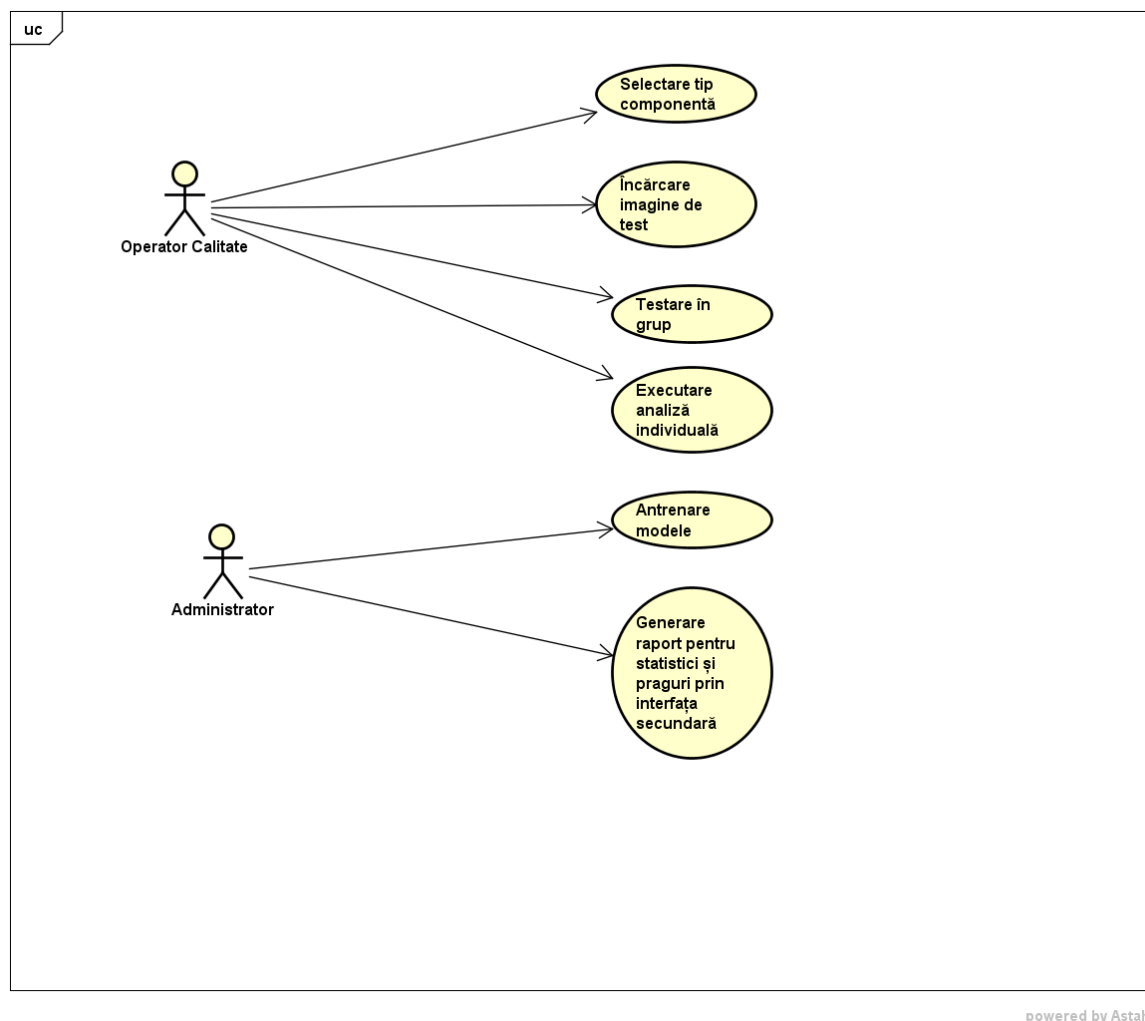
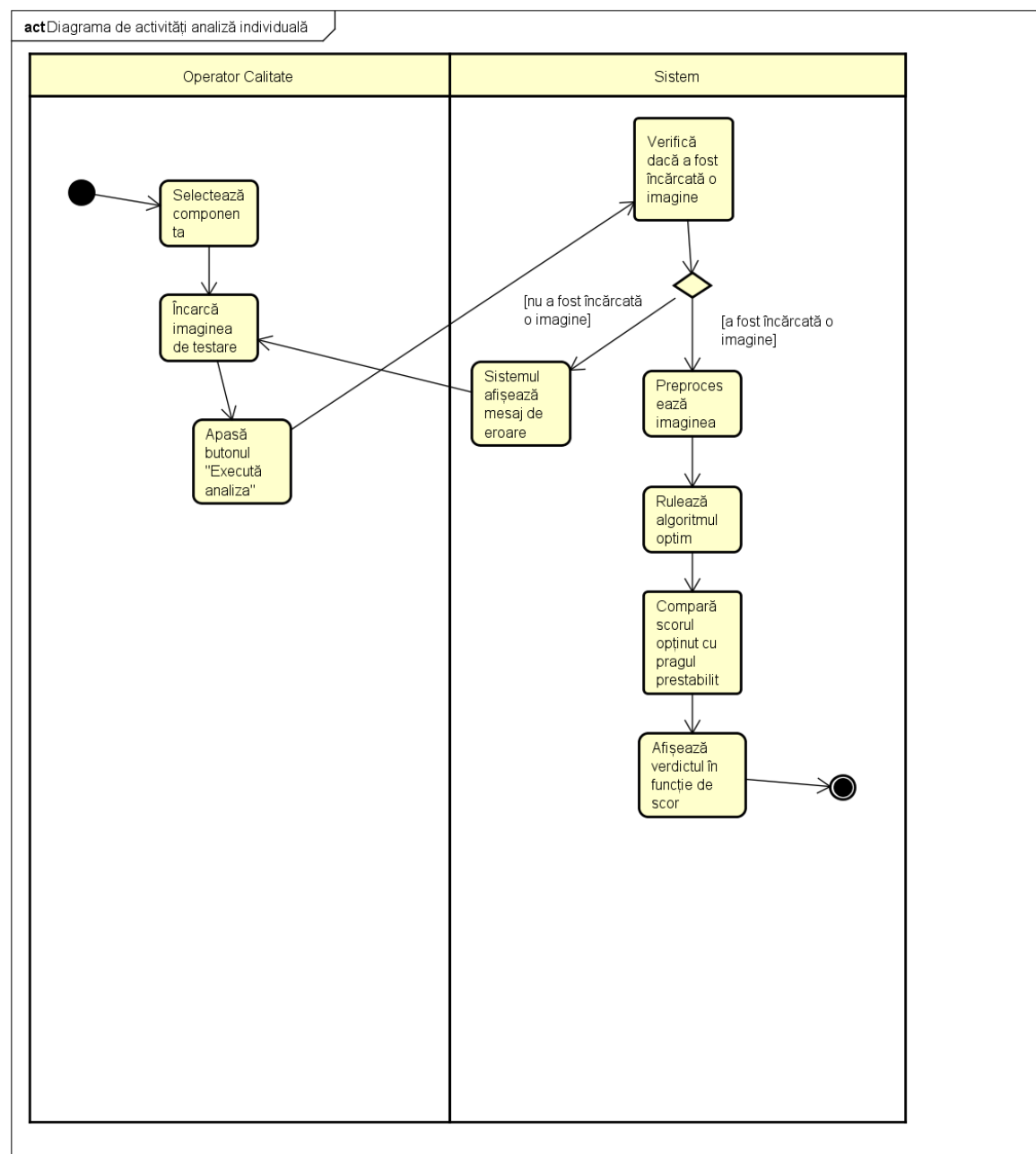
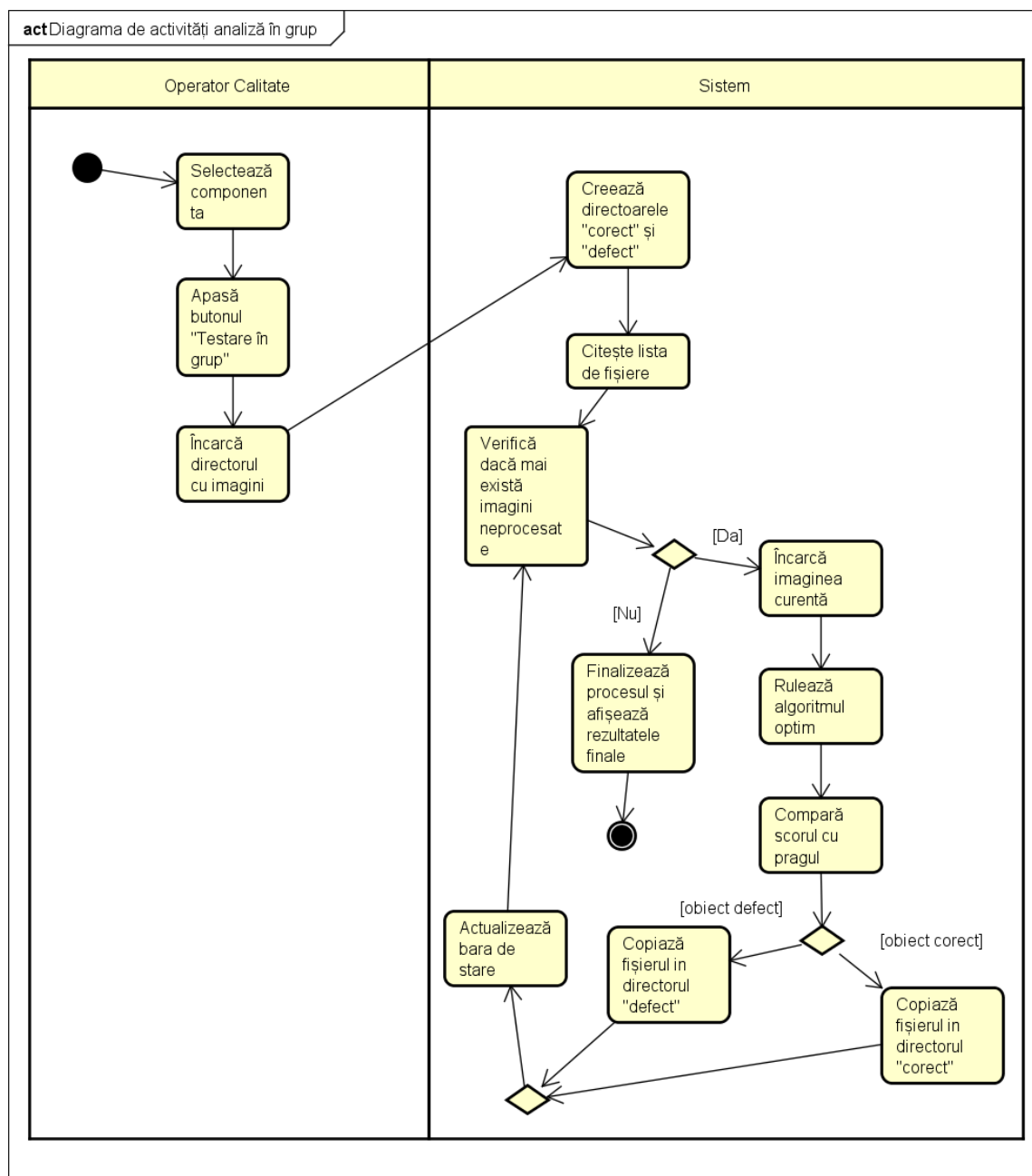


Figura 4.1: Diagrama cazurilor de utilizare



powered by Astah

Figura 4.2: Diagrama de activități analiză individuală



powered by Astah

Figura 4.3: Diagrama de activități analiză în grup

În momentul rulării codului, sistemul utilizează o procesare secvențială ce oferă un răspuns vizual imediat. Interfața grafică se separă de calculele matematice, iar modelele deja antrenate sunt încărcate în memoria plăcii video (sau pe procesor, dacă nu există placă video), fapt ce duce la transformări mai rapide ale imaginilor în matrici compatibile cu rețelele neuronale.

4.2 Procesarea datelor

Prelucrarea datelor înainte de etapa de testare reprezintă un pas critic pentru ca sistemul propus să funcționeze conform. Imaginile colectate din mediul industrial pot avea rezoluții variate, motiv pentru care toate imaginile sunt mai întâi redimensionate la mărimea de 256×256 pixeli cu ajutorul interpolării din biblioteca Pillow. Această rezoluție asigură atât păstrarea calității și a detaliilor imaginilor, cât și eficiența computațională și un consum mai scăzut de memorie GPU. Deoarece algoritmi implementați diferă din punct de vedere matematic, normalizarea inițială a imaginilor diferă.

Ahitectura propusă pentru Autoencoder este antrenată complet de la zero pe obiectele din seturile de date. Imaginile sunt scalate cu funcția `ToTensor()`, ce transformă fiecare pixel din intervalul $[0, 255]$ în $[0.0, 1.0]$. Această normalizare are rolul de a reduce erorile false cauzate de schimbări ale luminilor, umbre sau reflexii.

Algoritmi PaDiM și PatchCore sunt bazați pe surse externe, aceștia folosind rețele neuronale preantrenate cu care sunt extrase caracteristici ale imaginilor din seturile de date. Imaginile convertite în tensori PyTorch sunt normalizate cu media ($\mu = [0.485, 0.456, 0.406]$) și deviația standard ($\sigma = [0.229, 0.224, 0.225]$) pentru a aduce imaginile utilizate în același spațiu de caracteristici cu cel pe care a fost antrenat extractorul de trăsături.

Imaginile încărcate în interfață sunt citite cu ajutorul bibliotecii Pillow și convertite în format RGB, transformările PyTorch fiind ulterior aplicate în funcție de algoritmul selectat. În cadrul trecerii de la procesarea neuronală a datelor la afișarea lor în interfața grafică, hărțile termice generate sunt normalizate în intervalul $[0, 255]$ și convertite în imagini de tip `ImageTk.PhotoImage`.

4.3 Ingineria codului

Sistemul este proiectat respectând practicile de modularitate, separare a responsabilităților și programarea orientată pe obiecte. Această structură ușurează adăugarea ulterioară a unor noi algoritmi de detecție sau a unor categorii de piese noi.

Codul aplicației este structurat în module diferite, fiecare având un rol în funcționarea sistemului. Fiecare dintre cei trei algoritmi are un fișier de cod propriu (`autoencoder.py`, `padim.py` și `patchcore.py`) ce conține logica din spatele algoritmilor. Scriptul `calcul_praguri.py` deschide o interfață separată de sistemul principal, ce are ca scop calcularea și crearea unor fișiere text cu scorurile de eroare ale fiecărei imagini dintr-un director ales, în funcție de algoritmul selectat. Acest script este necesar în etapa de antrenare a sistemului pe un set nou de date pentru a ușura alegerea unui prag de eroare. Modulul `antrenare.py` execută partea de backend folosind scripturile algoritmilor și salvând aceste fișiere în cadrul aplicației, de unde pot fi ulterior extrase pentru etapa de testare. Scriptul `interfata.py` este partea de aplicație cu care interacționează utilizatorul pentru testarea propriu-zisă. În cadrul acestui script sunt adăugate toate componentele grafice și interactive, precum butoanele, bara de stare și fereastra în care apar rezultatele.

Sistemul propus prezintă o arhitectură de tip Model-View-Controller (MVC) ce a fost adaptată cerințelor învățării profunde. Interfața are rolul de prezentare (View) cu obiecte din `customtkinter` și de controler (Controller) datorită apelării metodei de predicție `self.prezice_image`. Modelele matematice (Model) sunt definite în scripturi izolate cu roluri diferite, ce pot fi extrase în funcție de nevoia sistemului în timpul rulării aplicației.

4.4 Managementul modelelor

Managementul modelelor constă în antrenarea controlată pe seturile de date disponibile, stocarea pe disc și maparea stărilor interne. În antrenare, procesul este realizat exclusiv pe componente conforme.

Antrenarea pentru Autoencoder este realizată în două sute de epoci cu ajutorul algoritmului de optimizare Adam[19], ce are o rată de învățare inițială de $\eta = 10^{-3}$. Sistemul calculează eroarea cu funcția de cost MSE (Eroarea Medie Pătratică) și o propagă înapoi pentru a ajusta ponderile. Pentru a evita stagnarea ratei de învățare, este implementat un scheduler de tip `ReduceLROnPlateau`, ce urmărește evoluția costului și reduce rata de învățare cu un factor de 0.1 dacă sistemul nu progresează timp de șapte epoci consecutive.

În cadrul algoritmului PaDiM, modelul rulează setul de date prin extractorul preantrenat ResNet18 și extrage caracteristici din primele trei straturi intermediare ale rețelei pentru a cuprinde atât detalii fine, cât și globale. Mai apoi este calculat pentru fiecare pixel vectorul de medie μ și matricea de covarianță Σ , iar în final inversa acestei matrici este serializată. Astfel este definită o distribuție Gaussiană multivariată a normalității pe disc.

Pentru algoritmul PatchCore, procesul este similar algoritmului PaDiM descris anterior: imaginile sunt trecute prin extractorul de caracteristici și sunt calculate hărțile locale de trăsături. Sistemul creează o bancă de memorie ce reține toate trăsăturile extrase din antrenare. Pentru a reduce mărimea acestei bănci de memorie, este utilizată o selecție Coreset Subsampling ce păstrează doar vectorii mai reprezentativi prin aproximare a distanțelor maxime. Astfel, spațiul utilizat de pe disc este redus, structura salvată fiind optimizată pentru interogări rapide.

Modelele pentru Autoencoder sunt stocate ulterior ca fișiere de stări PyTorch (`.pth`), în timp ce modelele PaDiM și PatchCore sunt serializate ca fișiere de tip Pickle (`.pkl`) pentru a păstra matricele de covarianță și băncile de memorie calculate. Pentru a combate problemele de compatibilitate ale structurilor, cum ar fi trecerea de la denumiri de variabile explicite precum `codificator_conv` la containere secvențiale, în cadrul interfeței au fost implementate mecanisme de re-mapare dinamică a cheilor (`incarca_stare_model`). Aceste mecanisme interceptează structura veche și o traduc în noul format cerut de clasă înainte de apelarea metodei încărcate.

Capitolul 5

Prezentarea sistemului propus

5.1 Interfața grafică

Deoarece sistemul propus este menit utilizării într-un context industrial, s-a urmărit ca interfața să fie simplă și ușor de folosit. Cu ajutorul bibliotecii CustomTkinter din Python, fereastra principală a fost configurată într-o culoare închisă, ce nu deranjează ochii, spre deosebire de culoarea albă. Fereastra a fost împărțită în trei secțiuni importante: panoul de comandă din partea stângă a ecranului, zona principală unde se poate vedea imaginea testată și rezultatul oferit de inteligența artificială după procesare, și bara de stare din partea de jos.

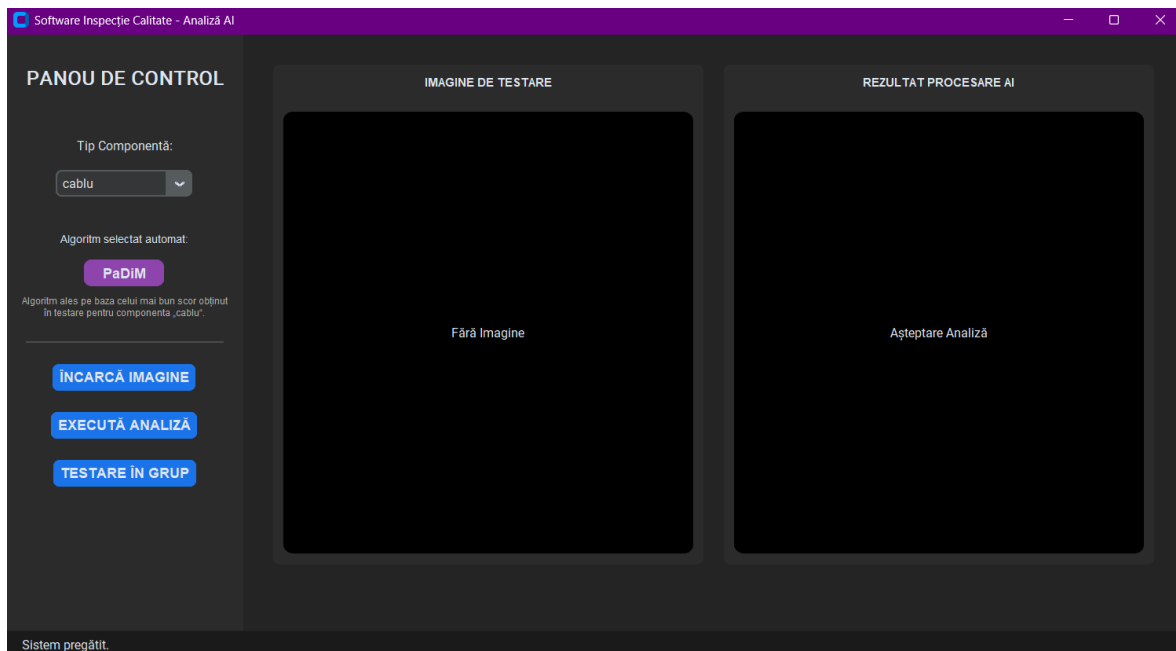


Figura 5.1: Fereastra în momentul deschiderii sistemului.

Această structură cu trei secțiuni respectă standardele moderne pentru interfețele sistemelor de tip SCADA (Supervisory Control and Data Acquisition) și terminalele HMI (Human-Machine Interface) folosite pe liniile de asamblare. Paleta cromatică întunecată a fost aleasă pentru a reduce efortul vizual al operatorilor ce monitorizează procesul de control al calității.

În panoul de comandă se găsesc toate elementele cu care utilizatorul poate interacționa. La început se poate alege dintr-o listă de componente pe care a fost antrenat modelul, iar în funcție de obiectul ales, sistemul propus schimbă automat algoritmul pentru a-l folosi mereu pe cel optim componentei respective. Fiecare algoritm are un fundal de culoare diferită pentru ca utilizatorul să poată face mai ușor distincția între aceștia. Din punct de vedere al codului, lista de componente este un obiect de tip `CTkComboBox` ce funcționează dinamic datorită parametrului `command`, ce a fost legat de o funcție numită `schimba_obiect_callback`.

Listing 5.1: Inițializarea listei pentru selectarea componentelor industriale

```
self.lbl_obiect = ctk.CTkLabel(self.sidebar_frame, text="
    Selectează Obiect:", anchor="w")
self.lbl_obiect.pack(fill="x", padx=20, py=(10, 0))

optiuni_obiecte = list(config_obiecte.keys())
self.combo_obiect = ctk.CTkComboBox(
    self.sidebar_frame,
    values=optiuni_obiecte,
    command=self.schimba_obiect_callback
)
self.combo_obiect.pack(fill="x", padx=20, py=10)
```

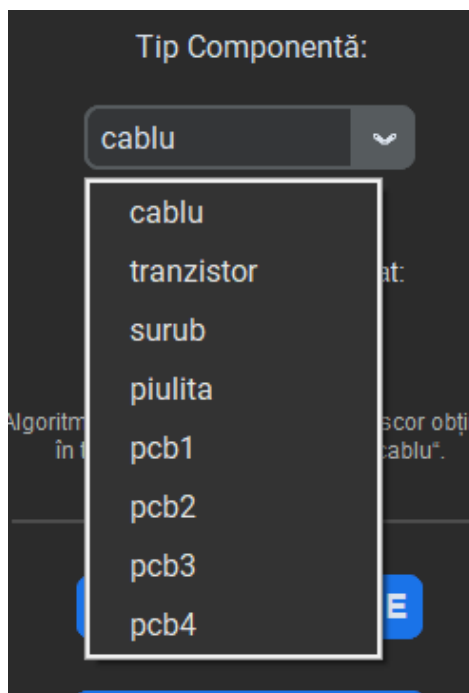


Figura 5.2: Lista cu componentele învățate de model.

Astfel, interfața monitorizează alegerea operatorului și își schimbă algoritmul afișat conform algoritmului optim din dicționarul de configurare.

Această automatizare a alegerii algoritmului este importantă deoarece operatorii nu trebuie să cunoască algoritmii sau noțiunile de Machine Learning pentru a putea utiliza sistemul. Sistemul alege algoritmii optimi pentru fiecare componentă dintr-un dicționar de configurare (`config_obiecte`).

Interfața este organizată cu ajutorul unor containere de tip `CTkFrame` poziționate cu funcția `pack()`. Această separare împiedică elementele precum bara de stare, panoul de control și zona principală a interfeței să se suprapună în momentul în care fereastra este redimensionată.

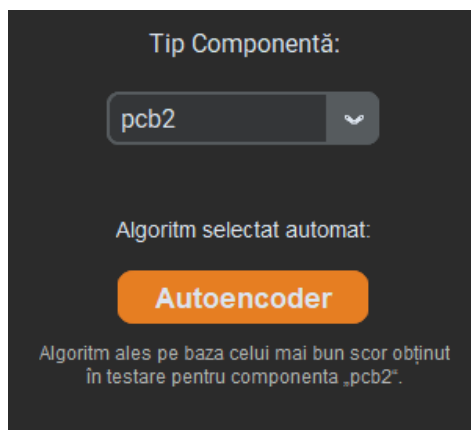


Figura 5.3: Interfață pentru Autoencoder

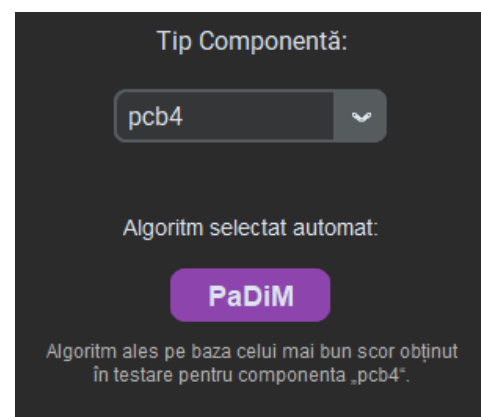


Figura 5.4: Interfață pentru PaDiM

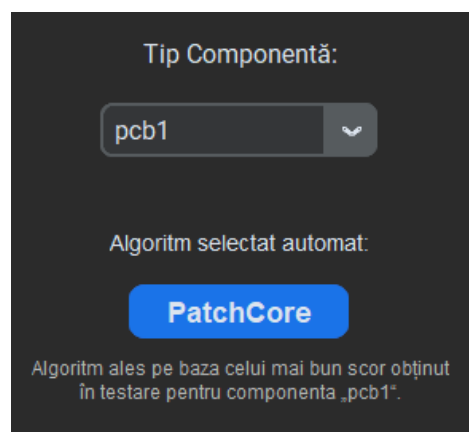


Figura 5.5: Interfață pentru PatchCore

Mai jos poate fi observată o descriere a celor trei butoane din cadrul interfeței grafice,

împreună cu explicații despre funcționalitatea lor:

1. **Încarcă imagine:** Acest buton deschide o fereastră nouă în directorul obiectului ales din lista de componente, de unde utilizatorul poate alege o imagine de test. Această fereastră este mereu deschisă direct în directorul componentei alese pentru a ușura procesul și pentru a preveni încărcarea unei imagini cu o altă componentă decât cea selectată.
2. **Execută analiză:** Butonul inițiază rularea sistemului pentru imaginea de test aleasă. În spate, sistemul preia imaginea, o convertește în tensor PyTorch, aplică normalizarea și algoritmul optim utilizat. În timpul rulării, interfața este blocată pentru a nu declanșa analize multiple în același timp.
3. **Testare în grup:** Acest buton este asemănător cu butonul „Încarcă imagine”, dar în loc să se aleagă o singură imagine, se selectează un întreg director de imagini ce vor fi sortate în imagini corecte și imagini defecte. Această funcție este necesară într-un flux de asamblare din interiorul unei fabrici, unde obiectele vin foarte rapid pe bandă. La final, sistemul copiază fișierele corecte și fișierele defecte în două directoare separate.

În cadrul sistemului propus, procesul de încărcare a imaginilor a fost optimizat printr-o filtrare dinamică a directorului inițial în funcție de componenta selectată în `CTkComboBox`. Această funcție a fost configurată pentru evitarea erorii umane, cum ar fi selectarea unei imagini cu un tranzistor în timp ce în listă a fost selectat un cablu.

Listing 5.2: Logica avansată de filtrare a directoarelor și gestionarea stărilor asincrone pentru elementele HMI

```
def incarca_image(self):
    from tkinter import filedialog
    cale_baza = r"E:\Facultate\code\Licenta\Aplicatie\dataset"
    obj = self.combo_obiect.get()

    init_dir = os.path.join(cale_baza, obj) if os.path.exists(os.path
        .join(cale_baza, obj)) else cale_baza

    self.cale_image = filedialog.askopenfilename(
        initialdir=init_dir,
        filetypes=[("Imagini", "*.png *.jpg *.jpeg")]
    )
    if self.cale_image:
        img_pil = Image.open(self.cale_image).convert("RGB")
        self.img_ctk_orig = ctk.CTkImage(light_image=img_pil, dark_image=
            img_pil, size=(450, 450))
        self.canvas_orig.configure(image=self.img_ctk_orig, text="")
```

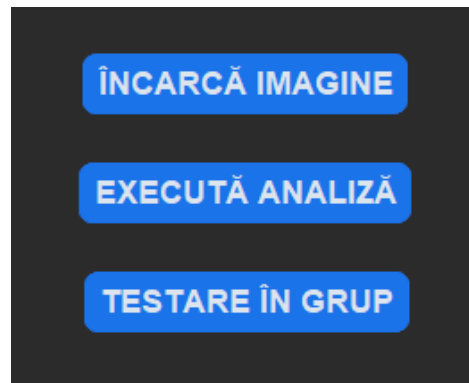


Figura 5.6: Cele trei butoane ale sistemului propus.

Modulul `filedialog` din biblioteca `tkinter` a fost introdus pentru a face posibilă deschiderea directoarelor și utilizarea imaginilor cu funcții precum `askopenfilename` și `askdirectory`. Butoanele sunt dezactivate temporar în timpul rulării pentru a preveni blocaje ce ar putea rezulta în urma încercării utilizatorului de a apăsa butoanele în mod repetat.

Listing 5.3: Dezactivarea elementelor de control

```
self.btn_executa.configure(state="disabled", text="Se proceseaza  
...")  
self.btn_incarca.configure(state="disabled")  
self.btn_grup.configure(state="disabled")
```

În partea dreaptă a ferestrei se află două cadrane: unul destinat imaginii de testare alese și unul în care putem observa rezultatul modelului după procesarea imaginii inițiale. Pentru algoritmul Autoencoder rezultatul este imaginea reconstruită cu inteligența artificială, în timp ce pentru PaDiM și PatchCore rezultatul este reprezentat de o hartă termică (heatmap), unde culoarea roșie indică zonele cu deviație mai ridicată. În jurul celui de-al doilea cadran este adăugată o margine colorată în funcție de statutul imaginii de test: verde pentru imagini corecte și roșu pentru imagini defecte.

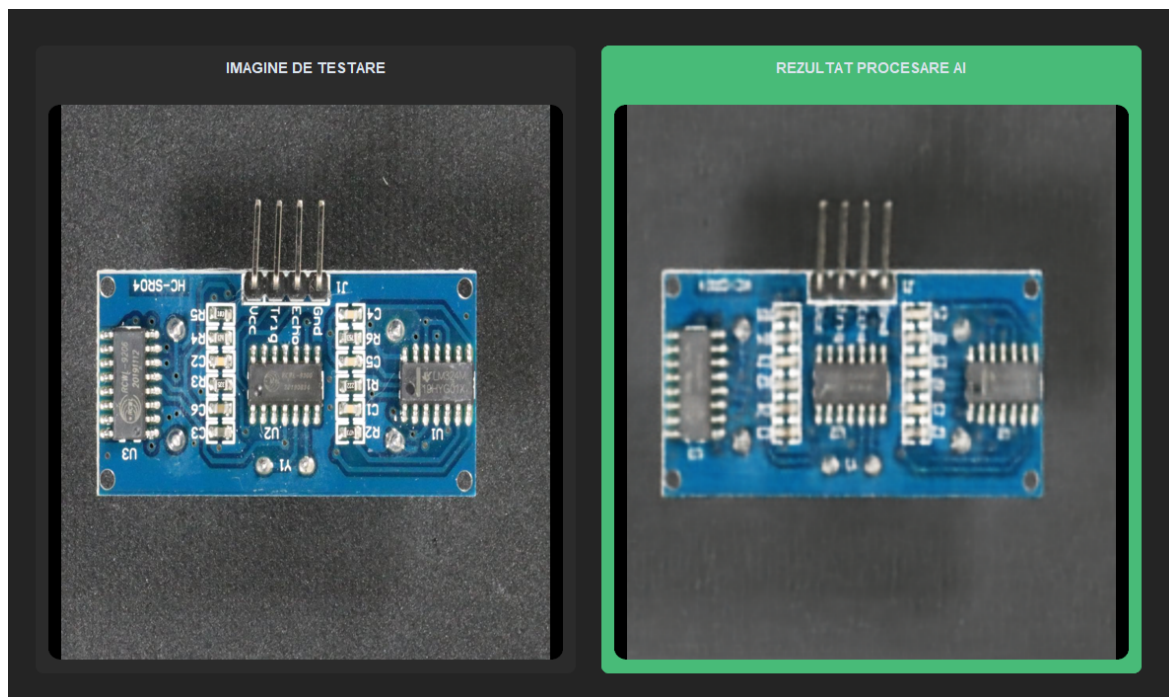


Figura 5.7: Rezultatele din interfață pentru reconstrucția imaginii originale cu o componentă corectă.

Această diferență vizuală este controlată de o variabilă booleană responsabilă pentru verdictul analizei. Astfel, în cazul unei piese conforme, containerul primește o nuanță de verde (#48bb78), iar pentru cazul contrar, când rezultatul depășește pragul setat pentru componenta respectivă, cadrul este transformat în roșu (#ff4d4d). În cazul algoritmilor bazați pe extracție de trăsături, interfața utilizează funcția pentru generarea hărților termice. Un astfel de exemplu poate fi observat în imaginea de mai jos:

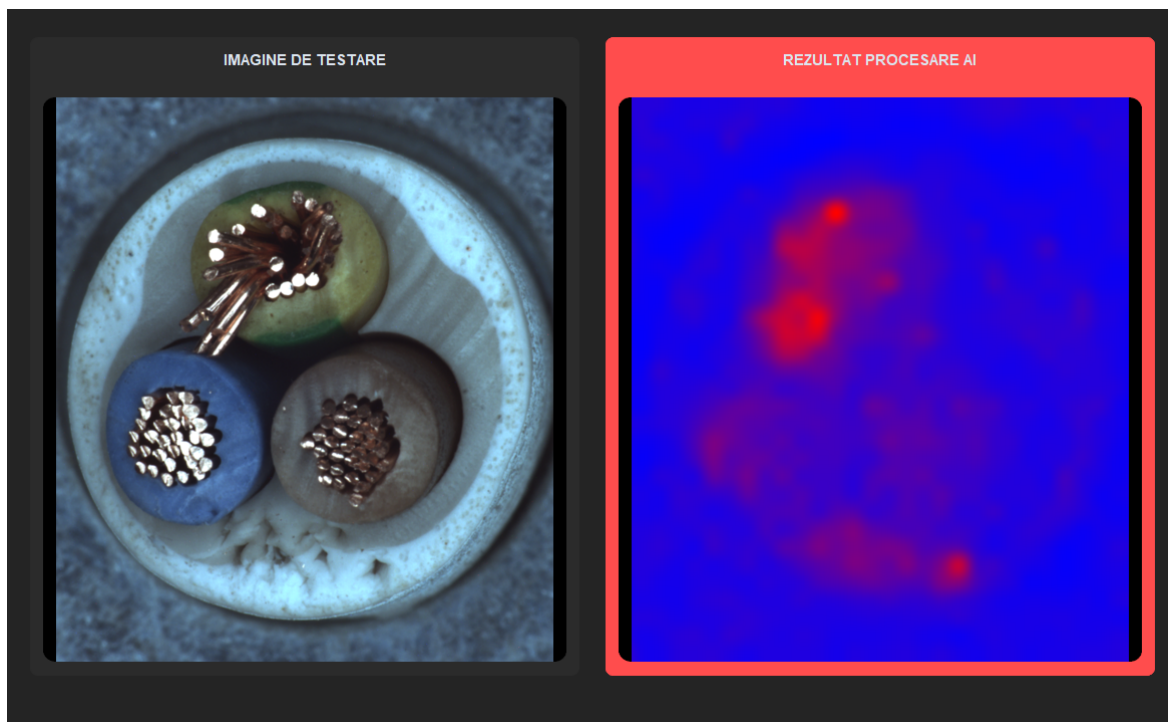


Figura 5.8: Rezultatele din interfață pentru un algoritm cu hartă termică cu o componentă defectă.

În partea de jos a ferestrei este bara de stare, ce clarifică acțiunea curentă a sistemului și oferă detalii despre procese. Bara de stare este menită să comunice cu utilizatorul în timpul execuției. În timpul testării individuale, aceasta oferă detalii legate de verdict, componenta aleasă și algoritmul utilizat în analiză. În schimb, pentru analiza unui întreg director de imagini, bara de stare are un rol mai important, deoarece ea afișează în timp real fiecare iterație a procesului de analizare. Astfel, utilizatorul nu vede doar o fereastră neschimbată și știe că sistemul funcționează. La finalul acestui proces, operatorul poate vedea în bara de stare câte imagini au fost corecte și câte defecte.

Listing 5.4: Actualizarea în timp real a bării de stare în timpul procesării în grup

```
for idx, f in enumerate(fisiere, 1):
    self.status_bar.configure(
        text=f"[{alg_nume}] Procesez {idx}/{len(fisiere)}: {f}",
        text_color="gray80")
    self.update_idletasks()
```

Se rulează Autoencoder pentru pcb2...

Figura 5.9: Bara de stare în timpul rulării unei singure imagini

Verdict: DEFECT | Algoritm folosit: Autoencoder | Componentă: pcb2

Figura 5.10: Bara de stare cu rezultatul unei rulări pentru o singură imagine.

[Autoencoder] Procesez 74/100: 059.JPG

Figura 5.11: Bara de stare în timpul rulării unui director

Grup finalizat cu PatchCore | 4 CORECTE, 96 DEFECTE.

Figura 5.12: Bara de stare cu rezultatul rularii unul întreg director.

5.2 Analiza individuală

În cadrul sistemului propus există două modalități de a analiza componentele. Prima dintre ele este analiza individuală, în care obiectele sunt alese pe rând, oferind de asemenea o analiză mai detaliată a obiectului în cadrul rezultatelor de după procesare. Această analiză începe odată ce utilizatorul a ales o imagine de test. Este implementat un sistem dinamic de încărcare a datelor prin intermediul unor instrucțiuni `if` din cadrul fișierului `interfata.py`.

Listing 5.5: Secvență de cod pentru inițializarea detectorilor și rularea algoritmilor optimi.

```
obj_num = self.combo_obiect.get()
alg_num = self.get_algoritm_optim(obj_num)
img_pil = Image.open(self.cale_imagine).convert("RGB")

self.status_bar.configure(
    text=f"Se rulează {alg_num} pentru {obj_num}...", text_color="gray80")
self.update_idletasks()

m_ae = self.get_model_ae(obj_num) if alg_num == "Autoencoder"
    else None
d_padim = None
if alg_num == "PaDiM":
    d_padim = PaDiMDetector(device=self.device)
    d_padim.load(f"model_{obj_num}_padim.pkl")
d_pc = None
if alg_num == "PatchCore":
    d_pc = PatchCoreDetector(device=self.device)
    d_pc.load(f"model_{obj_num}_patchcore.pkl")
```

Această abordare este necesară deoarece interpretarea rețelelor neuronale complexe este un proces intensiv din punct de vedere computațional. Funcția `update_idletasks()` a fost folosită deoarece, fără aceasta, interfața grafică bazată pe `customtkinter` s-ar bloca până la finalizarea analizei pe placa video (GPU).

Această condiție ușurează munca modelului, sistemul propus nefiind forțat să țină minte modelele de la fiecare algoritm în același timp. Metoda `self.update_idletasks()`, apelată înainte de decizia algoritmului, are scopul de a forța actualizarea interfeței.

După ce sistemul a pregătit datele, urmează preprocesarea imaginii într-un format compatibil cu cerințele algoritmului utilizat. Pentru Autoencoder, imaginile sunt redimensionate la 256×256 pixeli, iar valorile acestora sunt normalizate din $[0, 255]$ în $[0.0, 1.0]$ cu funcția `ToTensor()`, standard în PyTorch. În cazul algoritmilor PaDiM și PatchCore, imaginea este de asemenea normalizată, scăzând din fiecare pixel media (`mean`) și împărțind rezultatul la deviația standard (`std`).

Listing 5.6: Preprocesarea și transformarea imaginilor în tensori PyTorch

```
t_simple = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.ToTensor()
])

t_norm = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
        0.224, 0.225])
])

img_t      = t_simple(img_pil).unsqueeze(0).to(self.device)
img_t_norm = t_norm(img_pil).unsqueeze(0).to(self.device)
```

Funcția `unsqueeze(0)` adaugă o nouă axă tensorului la indexul 0, transformând matricea tridimensională a imaginii $[C \times H \times W]$ într-un element $[1 \times C \times H \times W]$. Acest pas este necesar deoarece structurile convoluționale și modulele de normalizare în PyTorch (`BatchNorm2d` sau `GroupNorm`) caută întotdeauna un tensor de ordin patru. În sistem a fost folosită o normalizare ce folosește constantele standard din ImageNet pentru a permite extractorului de trăsături ResNet18 să lucreze în același domeniu de distribuție geometrică pe care a fost preantrenat.

La finalul calculelor matematice realizate în cadrul algoritmului utilizat, modelul trebuie să traducă rezultatele în mediul vizual. În cazul algoritmilor PatchCore și PaDiM, sistemul generează o hartă termică unde albastru reprezintă zone cu deviație scăzută, iar roșu zone cu deviație ridicată. Matricea brută de distanțe statistice este interpolată biliniar prin funcții de eșantionare superioare până ajunge la dimensiunea originală a imaginii. Valorile scalate sunt în final suprapuse peste imaginea originală cu ajutorul unei palete de culori de tip JET[20] sau RGB, transformând variațiile subtile din spațiul metric în indicatori vizuali expliți.

Listing 5.7: Procedura de generare și suprapunere a hărții termice peste imaginea originală

```
def (self, anomaly_map_np):
    am_img = Image.fromarray((anomaly_map_np * 255).astype(np.uint8))
    am_img = am_img.resize((450, 450), resample=Image.BILINEAR)

    am_np = np.array(am_img)
    import cv2
    heatmap_cv = cv2.applyColorMap(am_np, cv2.COLORMAP_JET)
    heatmap_cv = cv2.cvtColor(heatmap_cv, cv2.COLOR_BGR2RGB)
    heatmap_pil = Image.fromarray(heatmap_cv)

    img_orig = Image.open(self.cale_image).convert("RGB").resize
        ((450, 450))
    blended = Image.blend(img_orig, heatmap_pil, alpha=0.5)

    return ctk.CTkImage(light_image=blended, size=(450, 450))
```

Pentru Autoencoder, rezultatul este tensorul imaginii reconstruite de rețeaua neuronală. Funcția `torch.clamp()` menține valorile tensorului strict între 0 și 1 pentru eliminarea unor posibile anomalii matematice. Matricea este separată de graful de calcul din memoria GPU și transferată în memoria sistemului gazdă prin `squeeze().detach().cpu()`, iar apoi `permute(1, 2, 0)` reordonează axele tensorului din [Canale, Înălțime, Lățime], format specific rețelelor neuronale, în [Înălțime, Lățime, Canale], format folosit pentru imagini. La final, fiecare pixel este înmulțit cu 255, iar matricea este convertită în întregi fără semn pe opt biți (`np.uint8`), pentru a permite redarea imaginii reconstruite.

Listing 5.8: Logica de procesare a rezultatelor în funcție de algoritmul selectat

```
if alg_num in ["PatchCore", "PaDiM"]:
    self.img_ctk_res = self.genereaza_heatmap(extra)
elif alg_num == "Autoencoder":
    res_np = (torch.clamp(extra.squeeze().detach().cpu(), 0, 1)
        .permute(1, 2, 0).numpy() * 255).astype(np.uint8)
    self.img_ctk_res = ctk.CTkImage(light_image=Image.fromarray(
        res_np), size=(450, 450))
```

5.3 Analiza unui grup

În cazul analizei unui grup de imagini, modelul nu mai primește doar o singură imagine, ci un întreg director de imagini. Această metodă de a analiza calitatea componentelor a fost implementată deoarece, într-un context real dintr-o fabrică, produsele vin una după alta pe bandă, sistemul fiind obligată să le proceseze și să treacă automat la următoarea. Analiza începe similar cu cea individuală, prin alegerea datelor ce trebuie încărcate în funcție de algoritm. Diferența în cazul analizei unui întreg director este că sistemul propus creează la început două noi directoare, „corecte” și „defecte”, în care imaginile urmează să fie sortate.

Listing 5.9: Crearea directoarelor pentru sortarea imaginilor corecte și a celor cu defecte

```
cale_corect = os.path.join(cale_folder, "corect")
cale_defect = os.path.join(cale_folder, "defect")
os.makedirs(cale_corect, exist_ok=True)
os.makedirs(cale_defect, exist_ok=True)
```

Această separare structurează rezultatele într-un mod ce poate fi direct utilizat într-un mediu industrial. Argumentul `exist_ok=True` din funcția `os.makedirs` verifică dacă directoarele de sortare au fost create deja într-o rulare anterioară, caz în care sistemul suprascrie pe acel director existent. Astfel sunt prevenite erorile generate din cauza suprapunerii.

Partea principală a analizei unui grup de imagini este reprezentat de o buclă în cadrul căreia fiecare imagine este evaluată folosind metoda `alează`, componentele fiind apoi sortate și contorizate, în bara de stare, în timp real. Pentru ca interfața să nu fie statică în timpul analizei fiecărui fișier, funcția `self.update_idletasks()` forțează actualizarea bării de stare după fiecare iterație. În buclă, modelul folosește variabila booleană `is_def` unde este salvat statutul imaginii (corectă sau defectă), apoi schimbă destinația (`dest`) conform rezultatului și copiază fișierul în acea cale (`shutil.copy`).

Listing 5.10: Bucla principală pentru analiza, sortarea și contorizarea în timp real a grupului de imagini

```
c_ok, c_def = 0, 0
for idx, f in enumerate(fisiere, 1):
    self.status_bar.configure(
        text=f"[{alg_num}] Procesez {idx}/{len(fisiere)}: {f}",
        text_color="gray80")
    self.update_idletasks()

    cale_f = os.path.join(cale_folder, f)
    img = Image.open(cale_f).convert("RGB")
    is_def, _ = self.prezice_imagine(img, alg_num, obj_num, m_ae,
                                     d_padim, d_pc)

    dest = cale_defect if is_def else cale_corect
    shutil.copy(cale_f, os.path.join(dest, f))
    if is_def:
        c_def += 1
    else:
        c_ok += 1

    self.status_bar.configure(
        text=f"Grup finalizat cu {alg_num} | {c_ok} CORECTE, {c_def} DEFECTE.",
        text_color="#48bb78")
    )
```

Procesul de analizare a unui grup de imagini este bazat pe o buclă de procesare ce tratează fiecare imagine pe rând până când toate fișierele au fost analizate. În sortare, a fost utilizată funcția `shutil.copy` în locul funcției `shutil.move` pentru a permite repetarea procesului pe același set de imagini.

5.4 Stabilirea pragurilor și a algoritmului optim

Alegerea pragurilor în funcție de algoritm și de obiect a reprezentat o provocare mai mare, deoarece nu există o metodă matematică universală, un algoritm dedicat sau o altă soluție predefinită pentru a le stabili. Singurul mod de a găsi acest prag este de a analiza scorul de anomalie al fiecărei imagini — atât conforme, cât și neconforme —, pentru a determina o valoare limită care să permită trecerea descriptorilor intacti, dar care, în același timp, să detecteze cu acuratețe piesele defecte.

Această dificultate în alegerea pragurilor provine din natura funcțiilor de cost și a indicatorilor de anomalie specifici fiecărei arhitecturi. Eroarea pentru Autoencoder este calculată prin intermediul erorii medii pătratice (MSE — Mean Squared Error) la nivel de pixel. Aceasta reprezintă media diferențelor la pătrat dintre imaginea originală și cea reconstruită în urma procesului de codificare și decodificare. Pentru algoritmi precum PaDiM și Patch-Core, scorul este stabilit cu ajutorul distanței statistice dintre trăsăturile imaginii de test și cele învățate de sistem în timpul antrenării. Din acest motiv, stabilirea unui prag de eroare global, valabil pentru orice tip de componentă, este imposibilă.

Pentru a ușura acest proces, a fost dezvoltat un sistem secundară de calcul al pragurilor. În cadrul acesteia se pot selecta algoritmul, componenta și directorul de imagini dorit, generându-se un fișier text care centralizează valorile minime, maxime și media erorilor, alături de o listă completă a tuturor imaginilor din director și a scorurilor asociate acestora.

Calculare Statistici si Praguri

1. Selecteaza Algoritmul:

Autoencoder

2. Selecteaza Obiectul:

cablu

3. Alege Folder Imagini

Niciun folder selectat

GENEREAZA RAPORT .TXT

Status: Pregatit

Figura 5.13: Interfața grafică a sistemului pentru generarea raportului de scoruri.

Sistemul dezvoltat se folosește de biblioteca `customtkinter` pentru interfața grafică. Utilizatorul selectează o componentă, un algoritm și un director de imagini pentru analiză, iar sistemul rulează imaginile, aplică normalizările și încarcă modelele salvate de tip `.pth` sau `.pkl` pentru a crea un raport cu eroarea fiecărei imagini din director, împreună cu minimul, maximul și media scorurilor. Analiza valorilor atât minime, cât și maxime din aceste rapoarte reprezintă un pas critic în alegerea unui prag de eroare echilibrat, care să reducă numărul de alarme false (erori de tip I) și, în același timp, să rămână extrem de eficient în detectarea defectelor reale (evitarea erorilor de tip II).

```
RAPORT STATISTICI SI SCORURI
Obiect: PCB1 | Algoritm: Autoencoder
Folder procesat: E:/Facultate/code/Licenta/Aplicatie/dataset/pcb1/test/Anomaly
-----
NR. IMAGINI: 100
MIN SCOR:    0.0021476690
MAX SCOR:    0.0098192701
MEDIE SCOR:  0.0041078802
-----
LISTA SCORURI INDIVIDUALE:
000.JPG -> 0.0036064270
001.JPG -> 0.0041942080
002.JPG -> 0.0075263879
003.JPG -> 0.0050097601
004.JPG -> 0.0098192701
005.JPG -> 0.0049637565
006.JPG -> 0.0056918235
007.JPG -> 0.0053367331
008.JPG -> 0.0043656426
009.JPG -> 0.0048756604
010.JPG -> 0.0031902574
011.JPG -> 0.0029559555
012.JPG -> 0.0034285891
013.JPG -> 0.0031343743
```

Figura 5.14: Exemplu de fișier text cu rezultatele scorurilor pentru PCB1 utilizând algoritmul Autoencoder.

După stabilirea pragurilor, următorul pas a fost verificarea rezultatelor și stabilirea algoritmului optim pentru fiecare componentă, deoarece niciun algoritm nu poate fi perfect pentru orice set de date. Am fost utilizată funcția de testare a unui grup de imagini pentru a nota pe rând rezultatele tuturor obiectelor, centralizându-le în tabelul de mai jos.

Testarea rezultatelor a presupus evaluarea separată a directoarelor cu imagini etichetate ca fiind „conforme” (corecte), respectiv „neconforme” (defecte). Astfel, a putut fi creată o matrice de confuzie pentru fiecare caz în parte, ce oferă o imagine de ansamblu clară asupra performanței fiecărui algoritm. Obiectivul principal a fost identificarea unui algoritm cât mai eficient și mai echilibrat, cu o rată ridicată de decizii corecte în analiza componentelor.

Deși a fost superior în majoritatea scenariilor, au existat trei cazuri specifice în care algoritmul PaDiM nu a obținut cele mai bune rezultate. Pentru plăcile de circuit imprimat (PCB) de tipul unu și trei, PatchCore a reprezentat algoritmul optim, analiza pentru placa unu fiind cea mai aproape de perfecțiune în controlul calității. De asemenea, pentru placa de circuit doi, deși PaDiM avea o rată de recunoaștere a imaginilor conforme mai ridicată, clasifica mai mult de jumătate din imaginile defecte ca fiind corecte (generând o rată mare de fals negative), motiv pentru care Autoencoderul a reprezentat opțiunea mai stabilă și mai sigură pentru mediul de producție.

Tabela 5.1: Rezultatele clasificării testelor pe obiecte și algoritmi cu algoritmi optimi colorați cu albastru.

Obiect	Algoritm	Corecte	Defecte
Cablu			
Fișiere conforme	Autoencoder	41	17
	PaDiM	51	7
	PatchCore	54	4
Fișiere neconforme	Autoencoder	32	60
	PaDiM	36	56
	PatchCore	45	47
Piuliță			
Fișiere conforme	Autoencoder	16	6
	PaDiM	20	2
	PatchCore	18	4
Fișiere neconforme	Autoencoder	62	32
	PaDiM	13	80
	PatchCore	39	54
Șurub			
Fișiere conforme	Autoencoder	38	3
	PaDiM	36	5
	PatchCore	36	5
Fișiere neconforme	Autoencoder	88	31
	PaDiM	34	85
	PatchCore	103	16
Tranzistor			
Fișiere conforme	Autoencoder	54	6
	PaDiM	58	2
	PatchCore	50	10
Fișiere neconforme	Autoencoder	19	21
	PaDiM	15	25
	PatchCore	14	25
PCB 1			
Fișiere conforme	Autoencoder	94	6
	PaDiM	90	10
	PatchCore	100	0
Fișiere neconforme	Autoencoder	33	67
	PaDiM	14	86
	PatchCore	4	96
PCB 2			
Fișiere conforme	Autoencoder	86	14
	PaDiM	98	2
	PatchCore	69	31
Fișiere neconforme	Autoencoder	21	79
	PaDiM	69	31
	PatchCore	34	66
PCB 3			
Fișiere conforme	Autoencoder	96	5

Continuare pe pagina următoare...

Obiect (Continuare)	Algoritm	Corecte	Defecte
	PaDiM	86	15
	PatchCore	83	18
Fișiere neconforme	Autoencoder	72	28
	PaDiM	50	50
	PatchCore	39	61
PCB 4			
Fișiere conforme	Autoencoder	92	8
	PaDiM	91	9
	PatchCore	85	15
Fișiere neconforme	Autoencoder	13	87
	PaDiM	5	95
	PatchCore	22	78

În urma analizării tabelului de mai sus, pot fi observate comportamentele distincte ale fiecărui algoritm utilizat. Pentru componentele cu o structură mai simplă și o textură similară pe toată suprafața lor, precum cablul, piulița, șurubul și tranzistorul, algoritmul PaDiM a fost cel optim. De exemplu, pentru piuliță, PaDiM a clasificat corect 20 din 22 de imagini conforme și 80 din 92 de imagini neconforme, aceste rezultate fiind cu mult superioare față de cele ale Autoencoderului, care a clasificat eronat 32 de imagini neconforme ca fiind corecte. Acest avantaj provine din capacitatea algoritmului PaDiM de a modela corelații spațiale prin intermediul matricei de covarianță.

În schimb, pentru plăcile de circuit imprimat, unde complexitatea vizuală este semnificativ mai crescută, PaDiM nu a mai fost cel mai eficient. Singura componentă din această categorie pentru care PaDiM a rămas optim a fost placa de circuit imprimat de tip patru, unde a detectat 95 din cele 100 de defecte.

În cazul plăcilor de tip unu și trei, algoritmul PatchCore a oferit rezultate mai bune. Pe primul set de circuite, PatchCore reușește să clasifice corect toate fișierele conforme și ratează doar 4 fișiere cu defecte, obținând cele mai bune rezultate din întreaga analiză. PatchCore a excelat în aceste situații datorită băncii de memorie cu trăsături optimizate (prin tehnica *Coreset Subsampling*).

Pentru al doilea tip de placă de circuit imprimat, dinamica analizei a fost diferită. Deși PaDiM a clasificat corect 98 din 100 de imagini conforme, acesta a ratat 69 de imagini neconforme — o rată de scăpare mult prea periculoasă într-un mediu industrial, unde livrarea unui produs defect generează costuri suplimentare logistice și de garanție. În comparație, deși Autoencoderul a identificat mai puține imagini conforme (generând mai multe alarme false), a clasificat corect 79 dintre cele 100 de imagini cu defecte. Astfel, s-a optat pentru un algoritm care, deși este mai strict și dă mai multe alarme false, reușește să identifice majoritatea defectelor reale. Această performanță se datorează faptului că Autoencoderul este antrenat exclusiv de la zero pe setul de date țintă, devenind extrem de sensibil la orice deviație structurală sau cromatică de la tiparul învățat.

Datele analizate mai sus demonstrează ipoteza inițială: nu există un algoritm general valabil care să funcționeze optim pe orice tip de componentă, alegerea arhitecturii fiind strâns legată de topologia și complexitatea vizuală a obiectului inspectat.

Dependența față de topologia obiectelor pune în evidență riscul supra-antrenării (*overfitting*) și capacitatea de generalizare a sistemului propus. Aceleași tipuri de obiecte mecanice și circuite sunt utilizate atât în antrenarea sistemului propus, cât și în etapa testării, fapt ce poate genera riscul ca modelele să memoreze zgomotul de fundal, reflexiile specifice sau unghiul fix de captură din setul de date. Pentru a combate acest risc, pragurile au fost alese nu doar pe baza imaginilor de antrenare, ci au fost optimizate în funcție de performanța lor asupra imaginilor de testare, asigurându-se astfel că sistemul propus detectează geometria reală a defectelor.

În concluzie, în urma analizei comparative poate fi observată o corelație între structura geometrică a componentei și eficiența algoritmilor utilizați. După sintetizarea rezultatelor obținute, s-a constatat că PatchCore este superior pentru structuri complexe și încărcate (plăcile de circuit imprimat), deoarece are capacitatea de a gestiona variațiile de textură. Pe de altă parte, pentru obiectele uniforme, PaDiM s-a dovedit a fi optim prin modelarea distribuției normale și utilizarea distanței Mahalanobis, care poate izola anomaliile fără costuri computaționale excesive.

Capitolul 6

Concluzii

6.1 Rezultate

Pentru a calcula performanța aplicației asupra fiecărei componente, s-a utilizat formula acurateții (ACC). Această formulă este definită ca suma predicțiilor adevărat pozitive și adevărat negative, împărțită la numărul total de obiecte care au fost clasificate.

$$ACC = \frac{AP + AN}{AP + AN + FP + FN} \times 100 \quad (6.1)$$

Unde:

1. AP (**Adevărat Pozitiv**) – imagini cu defecte pe care modelul le-a clasificat corect ca fiind defecte;
2. AN (**Adevărat Negativ**) – imagini conforme (corecte) pe care modelul le-a clasificat corect ca fiind conforme;
3. FP (**Fals Pozitiv**) – imagini conforme pe care modelul le-a clasificat greșit ca fiind defecte (alarme false);
4. FN (**Fals Negativ**) – imagini cu defecte pe care modelul le-a clasificat greșit ca fiind conforme (scăpări).

De asemenea, evaluarea unui sistem industrial necesită analiza metricilor de precizie (PRE) și sensibilitate (REC , numită și *recall*), care sunt estimate prin următoarele formule:

$$PRE = \frac{AP}{AP + FP} \times 100 \quad REC = \frac{AP}{AP + FN} \times 100 \quad (6.2)$$

Precizia reflectă capacitatea sistemului de a nu genera alarme false care să ducă la reinspectii inutile pe bandă, în timp ce sensibilitatea măsoară eficiența sistemului propus în detectarea defectelor reale, ratarea acestora reprezentând un factor critic de risc.

Pentru a menține calitatea producției în fluxul industrial, este necesară păstrarea unui echilibru optim între acești indicatori. Cea mai critică este rata de fals negative (FN), deoarece scăparea unui produs defect către clienți reprezintă atât costuri financiare directe (retururi, garanții), cât și penalizări la nivelul reputației și încrederii în companie. Pe de altă parte, o rată ridicată de alarme false (FP) încetinește considerabil fluxul de producție, operatorii umani fiind forțați să reinspecteze piese care sunt, în realitate, conforme. Astfel, metrica de acuratețe globală oferă o primă imagine de ansamblu asupra eficienței și funcționalității sistemului.

Tabela 6.1: Procentul de corectitudine (acuratețea globală) pentru fiecare obiect mecanic.

Obiect	Cablu	Piuliță	Șurub	Tranzistor
Acuratețe	71.33%	86.95%	75.62%	83.00%

Tabela 6.2: Procentul de corectitudine (acuratețea globală) pentru plăcile de circuit imprimat (PCB).

Obiect	PCB 1	PCB 2	PCB 3	PCB 4
Acuratețe	98.00%	82.50%	72.00%	93.00%

O analiză a acurateții globale indică diferențe clare între componentele cu o structură uniformă și componentele complexe, cu o topologie încărcată. În primul segment, piulița a înregistrat cea mai ridicată rată (86.95%), acuratețe ce provine din utilizarea algoritmului PaDiM, ce izolează defectele prin distanța Mahalanobis calculată direct pe hărțile de trăsături deep. În schimb, cablul a avut o acuratețe mai scăzută (71.33%) din cauza variabilității structurale generate de firele din interior și de torsionarea acestora, fapt ce a generat un număr mai mare de alarme false.

Pentru plăcile de circuit imprimat, clasele PCB 1 și PCB 4 au înregistrat o acuratețe de 98.00% și 93.00%, reprezentând un succes remarcabil al implementării. În cazul primei plăci, algoritmul PatchCore, bazat pe bănci de memorie cu eșantionare de tip coreset, a permis detectarea aproape perfectă a scurtcircuitelor, a lipsei de componente sau a întreruperilor de trasee, demonstrând o toleranță ridicată la zgomotul de fond structural al PCB-ului.

Deși a fost necesară utilizarea Autoencoderului ca alternativă mai stabilă, placa de tip doi a atins o acuratețe globală de 82.50%. Acest rezultat confirmă că maparea directă a

erorii pixel cu pixel (MSE loss) poate echilibra decizia de clasificare, chiar dacă penalizează performanța prin creșterea ușoară a alarmelor false. În final, pentru placa de tip trei, acuratețea scade la 72.00% din cauza structurii geometrice extrem de dense și a unor defecte micro, mult prea subtile pentru a fi evidențiate la rezoluția de lucru actuală.

6.2 Limitări ale sistemului

Deși rezultatele obținute indică faptul că sistemul propus este utilă pentru controlul calității, având decizii în mare parte corecte, sistemul prezintă o serie de limitări tehnice:

- 1. Dependența față de stabilirea manuală a pragului de decizie:** Așa cum s-a demonstrat în capitolul precedent, nu există o formulă matematică universală sau analitică pentru a calcula pragul de anomalie optim. Alegerea acestuia rămâne un proces de analiză statistică empirică ce necesită timp, efort uman și poate introduce o componentă de subiectivitate. Un prag prea permisiv va reduce alarmele false, dar va lăsa să treacă piese defecte (FN), în timp ce un prag prea strict va bloca fluxul de producție prin alarme false redundante (FP).
- 2. Sensibilitatea la variațiile mediului optic:** Algoritmii bazați pe extracția de trăsături geometrice și texturi sunt extrem de sensibili la condițiile de iluminare. Modificările fluxului luminos din fabrică, reflexiile parazite pe suprafețele metalice (șuruburi, piulițe) sau deviațiile unghiulare microscopice ale camerei de achiziție pot fi interpretate eronat ca anomalii structurale, ceea ce impune un mediu de captură strict controlat.
- 3. Cerințe hardware ridicate:** Rularea unor arhitecturi deep avansate (precum extractorul ResNet18 din spatele PaDiM și PatchCore) implică un cost computațional ridicat. Utilizarea unei unități de procesare grafică (GPU) performante este indispensabilă atât în faza de inferență individuală rapidă, cât și în cazul procesării în grup, pentru a se alinia cu timpii tact ai unei benzi industriale.
- 4. Limitarea dimensiunii imaginilor de intrare:** Redimensionarea standard la 256×256 pixeli penalizează detecția defectelor de dimensiuni microscopice. Creșterea rezoluției la 512×512 sau 1024×1024 pixeli ar rezolva această problemă, însă generează un blocaj de memorie (Out of Memory) pe echipamentele de calcul standard folosite în faza de dezvoltare.
- 5. Volumul limitat al datelor de antrenare:** Sistemul a fost antrenat și testat pe un număr redus de imagini pentru fiecare tip de obiect, seturile de date disponibile fiind reduse. Lipsa unor seturi de date masive poate limita capacitatea sistemului de învățare.
- 6. Lipsa validării pe seturi externe:** Sistemul nu a fost validat pe seturi provenite din alte sisteme, alte fabrici sau cu alte fundaluri. Testarea a fost realizată doar pe aceleași tipuri de obiecte, în aceleași condiții vizuale, astfel că schimbările majore de mediu nu au fost evaluate.

6.3 Perspective de viitor

Pentru îmbunătățirea ulterioară a performanțelor și extinderea aplicabilității practice a sistemului, sunt avute în vedere următoarele direcții de dezvoltare:

În primul rând, integrarea unui modul de optimizare automată a pragurilor în cadrul sistemului secundar, care să poată sugera o valoare optimă pe baza indicilor statistici din rapoartele text salvate, reducând efortul de calibrare manuală la introducerea unei noi componente.

În al doilea rând, optimizarea algoritmilor prin tehnici de cuantificare a modelelor (Quantization) și distilare a cunoștințelor (Knowledge Distillation) pentru a reduce complexitatea matematică a rețelelor neuronale. Acest pas ar asigura o inferență mult mai rapidă și ar permite rularea sistemului direct pe sisteme de calcul de tip embedded sau PC-uri industriale cu resurse hardware limitate.

Pe plan operațional, următorul pas logic îl reprezintă conectarea directă a interfeței grafice la automatele programabile (PLC) ce coordonează linia de asamblare. Astfel, funcția de analiză în grup nu se va limita la sortarea fișierelor în directoare, ci va transmite un semnal fizic de tip „Reject” către un actuator de pe bandă pentru eliminarea fizică imediată a piesei neconforme.

În final, o direcție academică importantă este implementarea tehnicilor de învățare continuă (Continual Learning). În versiunea actuală, modelele sunt statice după finalizarea antrenării. Dezvoltarea unui loop de feedback, în care alarmele false corectate de un operator uman să fie folosite pentru reantrenarea incrementală a modelelor, ar garanta auto-calibrarea și creșterea progresivă a acurateței direct în mediul de lucru.

6.4 Concluzie

Lucrarea de față a abordat o problemă de mare actualitate în contextul automatizării și modernizării proceselor industriale, reușind realizarea unui pachet software complet pentru controlul automat al calității. Proiectul îmbină tehnologii robuste din sfera inteligenței artificiale, utilizând bibliotecile PyTorch și Torchvision pentru procesarea tensorială din backend, cu un mediu de operare ergonomic și intuitiv dezvoltat în CustomTkinter.

Un punct forte al sistemului propus îl reprezintă flexibilitatea sa arhitecturală: acesta oferă acces la mai multe paradigme de detectare a anomaliilor (Autoencoder, PaDiM, Patch-Core), permițând maparea algoritmului optim pe specificul geometric al fiecărei piese. Cele două moduri de funcționare — analiza individuală și analiza în grup — fac software-ul perfect adaptabil atât pentru investigații detaliate de laborator, cât și pentru procesarea rapidă specifică fluxurilor de producție în masă.

Rezultatele experimentale validează utilitatea practică a sistemului. Deși calibrarea pragurilor a reprezentat o etapă analitică complexă, validarea finală a demonstrat o eficiență ridi-

cată, evidențiată prin acuratețea de 98.00% obținută pe primul set de plăci de circuit imprimat (PCB 1).

Din punct de vedere al ingineriei software, structura modulară a codului (`autoencoder.py`, `padim.py`, `patchcore.py`) confirmă avantajele programării orientate pe obiecte în aplicațiile de Deep Learning. Decuplarea completă a logicii de inferență de interfața grafică permite mentenanța facilă a sistemului, actualizarea independentă a modelelor matematice și oferă scalabilitatea necesară pentru integrarea unor noi arhitecturi neuronale în viitor.

În concluzie, sistemul realizat reprezintă o soluție viabilă și performantă pentru digitalizarea inspecției vizuale în fabrici, demonstrând că un algoritm cu un nivel ridicat de complexitate matematică poate fi încapsulat într-o interfață simplă, accesibilă oricărui operator din mediul industrial.

Bibliografie

- [1] E. Pelican. Algoritmi de calcul științific (recunoașterea formelor). Note de curs, Facultatea de Matematică și Informatică, Universitatea Ovidius din Constanța, 2025.
- [2] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 8024–8035, 2019.
- [3] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.
- [4] Alan D. Moore. *Python GUI Programming with Tkinter: Develop responsive and powerful GUI applications with Tkinter and Python*. Packt Publishing Ltd, Birmingham, UK, 2018.
- [5] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.
- [6] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [7] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020.
- [8] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, USA, 2016. <https://www.deeplearningbook.org>.
- [9] Thomas Defard, Aleksandr Setkov, Angelique Loesch, and Romaric Audigier. Padim: a patch distribution modeling framework for anomaly detection and localization. In

- International Conference on Pattern Recognition (ICPR) Workshops*, pages 475–489. Springer, 2021.
- [10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, pages 770–778, 2016.
- [11] Prasanta Chandra Mahalanobis. On the generalized distance in statistics. *Proceedings of the National Institute of Sciences of India*, 2:49–55, 1936.
- [12] Karsten Roth, Latha Pemberton, Thomas Le, Samet Akcay, Thomas Beluch, and Enkelejda Kasneci. Towards total recall in industrial anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14318–14328, 2022.
- [13] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, New Orleans, LA, USA, 2019.
- [14] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, Toulon, France, 2017.
- [15] Yuxin Wu and Kaiming He. Group normalization. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3–12, 2018.
- [16] P.Bergmann, M.Fauser, D.Sattlegger, and C.Steger. The MVTec anomaly detection dataset: A comprehensive real-world dataset for unsupervised anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9592–9600, 2019.
- [17] Yang Zou, Jongheon Jeong, Latha Pemula, Dongqing Zhang, and Onkar Dabeer. Spot-the-difference self-supervised pre-training for anomaly detection and segmentation. *arXiv preprint arXiv:2207.14315*, 2022.
- [18] C. M. Puchianu. Ingineria sistemelor software. Note de curs, Facultatea de Matematică și Informatică, Universitatea Ovidius din Constanța, 2025.
- [19] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, San Diego, CA, USA, 2015.
- [20] Gary Bradski. The OpenCV library. *Dr. Dobb’s Journal of Software Tools*, 25(11):120–125, 2000.